

MD005 - Sistemes Basats en el Coneixement

Segona entrega: Classificació

Professora: Jordina Torrents

Data d'entrega: 22 de gener de 2026

Nom de l'alumne: Gerard Pascual Fontanilles

Informació sobre el conjunt de dades

Per fer aquest treball he utilitzat el conjunt de dades de pokemons, ja que em sembla un tema interessant i divertit per treballar. S'ha extret de:

<https://www.kaggle.com/datasets/hamzacyberpatcher/data-of-1010-pokemons>

El conjunt de dades conté informació sobre 1025 pokemons diferents, incloent-hi les seves estadístiques bàsiques, tipus, generació i si evolucionen d'algun altre pokemon.

Les característiques principals del conjunt de dades són:

- Pokemon's id
- Pokemon's name
- Pokemon's rank (whether it is legendary, mythical, baby or an ordinary pokemon)
- Pokemon's generation
- The pokemon from which this pokemon evolved from
- It's primary and secondary type as type1 and type2
- It's base stats
- It's height (decimetres) and weight (hectograms)
- It's abilities (Note: if a pokemon has multiple abilities such as overgrow and chlorophyll then they are separated by a space and if there is a space in the name of the ability it has been replaced by a hyphen for example a pokemon with abilities blaze and solar power it is given as blaze solar-power)
- A little description of the pokemon in english (description of pokemons from 1009 is marked as "Not Available").

Imports necessaris

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, f1_score, confusion_matrix
import math
from sklearn.model_selection import GridSearchCV
import pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import precision_recall_curve
from sklearn.multiclass import OneVsRestClassifier
import seaborn as sns
from imblearn.over_sampling import SMOTE
```

```
from collections import Counter
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```
In [2]: df = pd.read_csv('pokemons.csv')
```

```
In [3]: df.head()
```

```
Out[3]:
```

	id	name	rank	generation	evolves_from	type1	type2	hp	atk	def	spatk	spdef	sp
--	----	------	------	------------	--------------	-------	-------	----	-----	-----	-------	-------	----

0	1	bulbasaur	ordinary	generation- i	nothing	grass	poison	45	49	49	65	65	
---	---	-----------	----------	------------------	---------	-------	--------	----	----	----	----	----	--

1	2	ivysaur	ordinary	generation- i	bulbasaur	grass	poison	60	62	63	80	80	
---	---	---------	----------	------------------	-----------	-------	--------	----	----	----	----	----	--

2	3	venusaur	ordinary	generation- i	ivysaur	grass	poison	80	82	83	100	100	
---	---	----------	----------	------------------	---------	-------	--------	----	----	----	-----	-----	--

3	4	charmander	ordinary	generation- i	nothing	fire	NaN	39	52	43	60	50	
---	---	------------	----------	------------------	---------	------	-----	----	----	----	----	----	--

4	5	charmeleon	ordinary	generation- i	charmander	fire	NaN	58	64	58	80	65	
---	---	------------	----------	------------------	------------	------	-----	----	----	----	----	----	--



```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    1025 non-null   int64
1   name                  1025 non-null   object
2   rank                  1025 non-null   object
3   generation            1025 non-null   object
4   evolves_from          1025 non-null   object
5   type1                 1025 non-null   object
6   type2                 526 non-null    object
7   hp                    1025 non-null   int64
8   atk                   1025 non-null   int64
9   def                   1025 non-null   int64
10  spatk                 1025 non-null   int64
11  spdef                 1025 non-null   int64
12  speed                 1025 non-null   int64
13  total                 1025 non-null   int64
14  height                1025 non-null   int64
15  weight                1025 non-null   int64
16  abilities             1025 non-null   object
17  desc                  1025 non-null   object
dtypes: int64(10), object(8)
memory usage: 144.3+ KB
```

Primer de tot eliminem les columnes que no aporten informació rellevant per al model, com ara `desc`, `id`, `name` i `total`.

Descartem `desc`, `id`, `name` perquè no aporten informació útil per a la classificació (`name` i `id` van relacionats perquè mirant la pokédex es pot saber quin pokémon és pel seu número de fila) i `total` és redundant ja que és la suma de les altres estadístiques i podria introduir multicolinealitat.

```
In [5]: df.drop(columns=['desc', 'id', 'name', 'total'], inplace=True)
```

```
In [6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   rank                  1025 non-null   object
1   generation            1025 non-null   object
2   evolves_from          1025 non-null   object
3   type1                 1025 non-null   object
4   type2                 526 non-null    object
5   hp                    1025 non-null   int64
6   atk                   1025 non-null   int64
7   def                   1025 non-null   int64
8   spatk                 1025 non-null   int64
9   spdef                 1025 non-null   int64
10  speed                 1025 non-null   int64
11  height                1025 non-null   int64
12  weight                1025 non-null   int64
13  abilities             1025 non-null   object
dtypes: int64(8), object(6)
memory usage: 112.2+ KB
```

Veiem com ara conjunt de dades realment té 1025 entrades (Pokémons), com ens deien a la descripció del conjunt de dades i 13 columnes (característiques), després d'haver eliminat les que no aportaven informació rellevant.

Totes semblen estar completes excepte la columna `type2`, que té valors nuls (NaN), suposem que és per als pokémons que només tenen un tipus.

Procedim a comprovar-ho:

```
In [7]: null_counts = df.isnull().sum()  
print(null_counts[null_counts > 0])
```

```
type2      499  
dtype: int64
```

Com es pot veure, només la columna `type2` té valors nuls (NaN). Això té sentit, ja que alguns pokémons només tenen un tipus (`type1`) i no tenen un segon tipus (`type2`).

De moment omplim els nuls de `type2` amb 'None' per indicar que no tenen segon tipus. Després analitzarem més a fons la columna per veure si fa falta eliminar-la o transformar-la.

```
In [8]: df['type2'] = df['type2'].fillna('None')
```

Mirant la resta de variables, la columna `evolves_from`, veiem que no hi ha cap valor nul (NaN), cosa que és estranya. Ja que alguns pokémons no evolucionen d'altres, haurien de tenir valors nuls (NaN) en aquesta columna. Ho comprovem:

```
In [9]: print(df['evolves_from'].unique()[0:10])
```

```
['nothing' 'bulbasaur' 'ivysaur' 'charmander' 'charmeleon' 'squirtle'  
'wartortle' 'caterpie' 'metapod' 'weedle']
```

Com podem veure, la columna `evolves_from` no té valors nuls (NaN), sinó que utilitza la paraula "nothing" per indicar que un pokémon no evoluciona d'altre. Això explica per què `df.info()` ens deia que no hi havia nuls en aquesta columna.

```
In [10]: print(df['evolves_from'].value_counts().head())
```

```
evolves_from  
nothing      541  
eevee         8  
tyroque       3  
applin        3  
scyther       2  
Name: count, dtype: int64
```

Com podem veure, hi ha molts pokémons que no evolucionen d'altre (valor "nothing"). Creem una nova columna que simplifiqui aquesta informació, al model normalment li importa més si el pokémon és una forma base o no, que saber exactament de qui ve.

Ja que si utilitzem aquesta columna tal qual, el model tractarà 'nothing' com una categoria més. Això està bé, però té un problema: la columna tindrà centenars de categories diferents (una per a cada pokémon que evoluciona), cosa que no es gens òptima per la majoria de models.

Definim una nova columna binària `is_base` que indiqui si un pokémon és una forma base (1- no evoluciona de ningú) o no (0- evoluciona d'un altre pokémon).

```
In [11]: df['is_base'] = df['evolves_from'].apply(lambda x: 1 if x == 'nothing' else 0)  
display(df[['evolves_from', 'is_base']].head())
```

	evolves_from	is_base
0	nothing	1
1	bulbasaur	0
2	ivysaur	0
3	nothing	1
4	charmander	0

Hem comprovat que la nova columna `is_base` funciona correctament.

Ara eliminem la columna original 'evolves_from', ja que no la farem servir.

```
In [12]: df.drop(columns=['evolves_from'], inplace=True)
```

```
In [13]: df.head()
```

```
Out[13]:
```

	rank	generation	type1	type2	hp	atk	def	spatk	spdef	speed	height	weight	abilities
0	ordinary	generation-i	grass	poison	45	49	49	65	65	45	7	69	overgrow chlorophyll
1	ordinary	generation-i	grass	poison	60	62	63	80	80	60	10	130	overgrow chlorophyll
2	ordinary	generation-i	grass	poison	80	82	83	100	100	80	20	1000	overgrow chlorophyll
3	ordinary	generation-i	fire	None	39	52	43	60	50	65	6	85	blaze solar power
4	ordinary	generation-i	fire	None	58	64	58	80	65	80	11	190	blaze solar power

Abans de continuar, fem una comprovació final de valors nuls (NaN) en tot el DataFrame:

```
In [14]: print(df.isnull().sum().sum())
```

0

Perfecte, no hi ha més valors nuls. Ara comprovem si hi ha files duplicades:

```
In [15]: duplicated_count = df.duplicated().sum()
print(f"Número de files duplicades: {duplicated_count}")
```

Número de files duplicades: 0

Tenim 0 files duplicades. Ara ja podem fer una revisió ràpida de les dades per detectar possibles errors o inconsistències.

Comprovem primer les columnes numèriques calculant estadístiques descriptives bàsiques:

```
In [16]: df.describe()
```

Out[16]:

	hp	atk	def	spatk	spdef	speed	height
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	70.184390	77.521951	72.507317	70.080976	70.205854	67.186341	12.116098
std	26.631054	29.782541	29.286972	29.658378	26.639329	28.717227	12.481673
min	1.000000	5.000000	5.000000	10.000000	20.000000	5.000000	1.000000
25%	50.000000	55.000000	50.000000	47.000000	50.000000	45.000000	5.000000
50%	68.000000	75.000000	70.000000	65.000000	67.000000	65.000000	10.000000
75%	85.000000	100.000000	90.000000	90.000000	86.000000	88.000000	15.000000
max	255.000000	181.000000	230.000000	173.000000	230.000000	200.000000	200.000000



L'anàlisi descriptiu (`describe`) confirma la necessitat de preprocessament abans d'entrenar els models:

1. Totes les columnes tenen un `count` de 1025 i els valors mínims són lògics (cap valor 0 o negatiu en estadístiques), confirmant una vegada més que no hi ha valors nuls amagats.
2. Existeix una diferència massiva de magnituds entre variables. Mentre que l' `atk` o la `def` es mouen en rangs de 5 a 250, el `weight` (pes) arriba fins a gairebé 10.000.
3. Tant en pes com en alçada, el valor màxim està molt allunyat del 75è percentil, indicant la presència de valors extrems (Pokémons gegants o molt pesats).

Aquesta disparitat de rangs justifica l'aplicació obligatòria d'un **escalat de dades** (`StandardScaler`). Sense ell, variables com el pes dominarien injustament el càlcul de distàncies en algoritmes eclipsant variables crítiques però de menor magnitud com la velocitat.

Tot i així, abans de seguir endavant, fem una anàlisi visual generant box plots de les variables numèriques per detectar possibles outliers que podrien distorsionar els models.

```
In [17]: numeric_cols = df.select_dtypes(include=['number']).columns.tolist()
plot_cols = [c for c in numeric_cols]

num_plots = len(plot_cols)
cols = 3
rows = math.ceil(num_plots / cols)

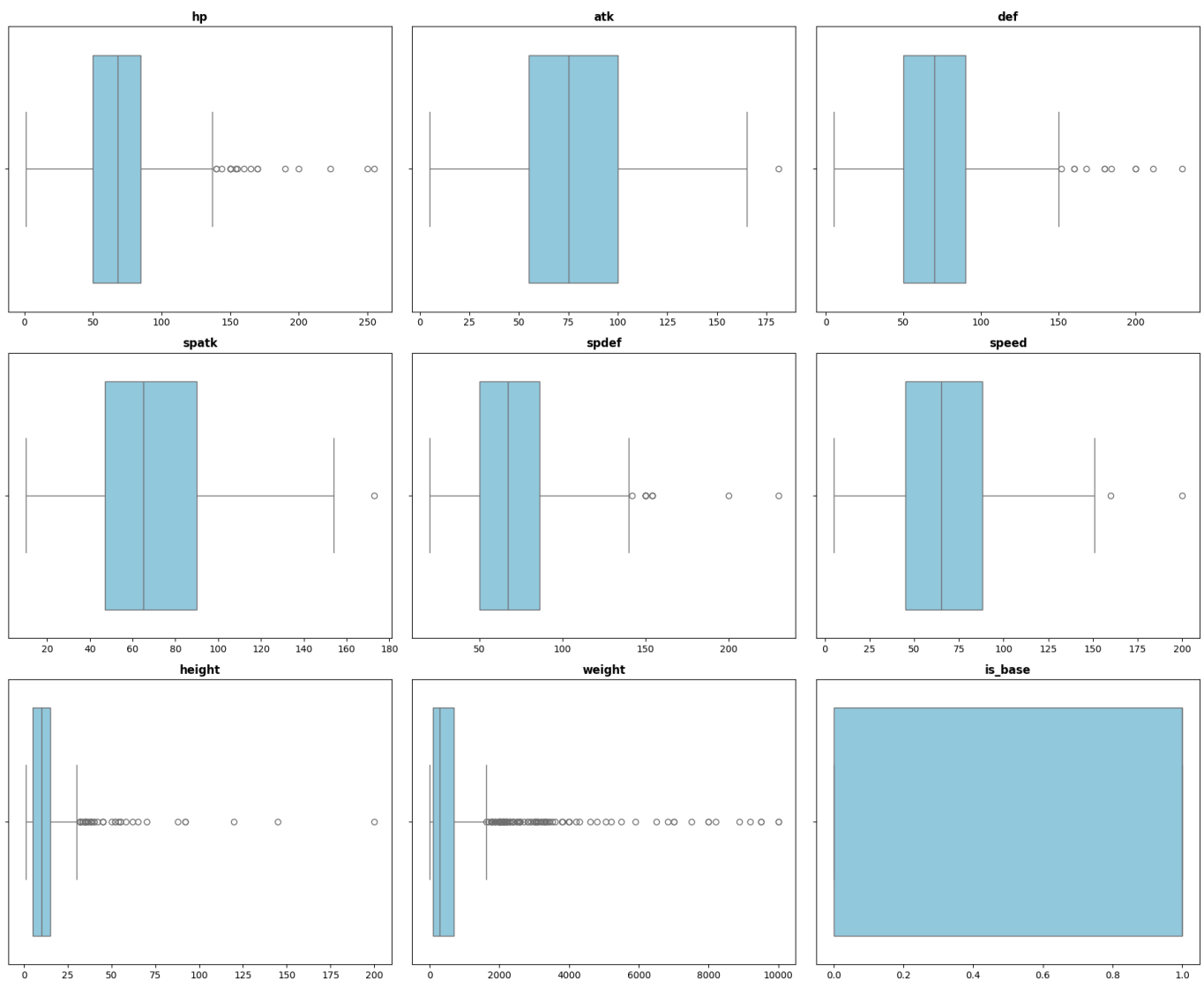
fig, axes = plt.subplots(rows, cols, figsize=(18, 5 * rows))
axes = axes.flatten()

for i, col in enumerate(plot_cols):
    sns.boxplot(data=df, x=col, ax=axes[i], color='skyblue')

    axes[i].set_title(f'{col}', fontweight='bold')
    axes[i].set_xlabel('')

for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.suptitle('Anàlisi d\'Outliers', fontsize=16, y=1.02)
plt.tight_layout()
plt.show()
```



També executem els 5 pokémons més extrems en pes, vida (hp) i velocitat per identificar quins són aquests outliers i si són errors o pokémons reals.

```
In [18]: print("TOP 5 POKÉMON MÉS PESATS")
display(df[['weight', 'rank', 'generation']].nlargest(5, 'weight'))
print("\nTOP 5 POKÉMON AMB MÉS VIDA (HP)")
display(df[['hp', 'rank', 'generation']].nlargest(5, 'hp'))
print("\nTOP 5 POKÉMON MÉS RÀPIDS")
display(df[['speed', 'rank', 'generation']].nlargest(5, 'speed'))
```

TOP 5 POKÉMON MÉS PESATS

	weight	rank	generation
789	9999	legendary	generation-vii
796	9999	ordinary	generation-vii
382	9500	legendary	generation-iii
889	9500	legendary	generation-viii
749	9200	ordinary	generation-vii

TOP 5 POKÉMON AMB MÉS VIDA (HP)

	hp	rank	generation
241	255	ordinary	generation-ii
112	250	ordinary	generation-i
798	223	ordinary	generation-vii
894	200	legendary	generation-viii
201	190	ordinary	generation-ii

TOP 5 POKÉMON MÉS RÀPIDS

	speed	rank	generation
893	200	legendary	generation-viii
290	160	ordinary	generation-iii
794	151	ordinary	generation-vii
100	150	ordinary	generation-i
385	150	mythical	generation-iii

Tots els pokémons són reals, no hi ha errors evidents en les dades. Els outliers detectats (com *Celesteela* [797 de la Pokédex] o *Cosmoem* [790 de la Pokédex, ex. <https://www.pokemon.com/us/pokedex/cosmoem>]) són pokémons legítims coneguts per les seves estadístiques extremes. Això és coherent amb la naturalesa del dataset, on els pokémons llegendaris i alguns normals poden tenir estadístiques molt altes.

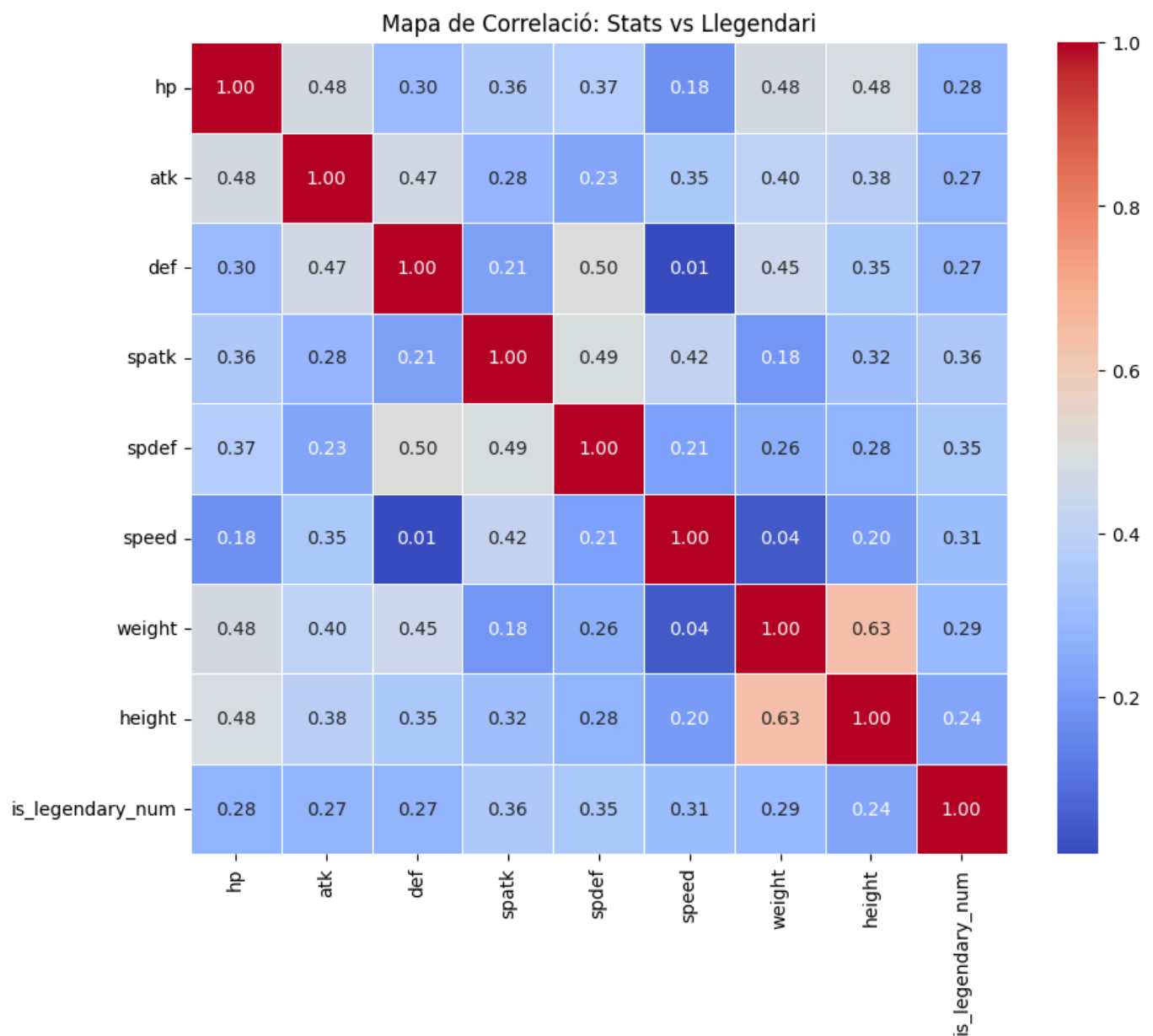
Per tant, no eliminarem cap outlier, ja que formen part integral del domini i poden aportar senyal valuós per a la classificació.

També generem un mapa de calor de correlacions per veure com es relacionen les estadístiques numèriques entre si i amb la variable **rank** (si és llegendari i mític o no).

```
In [19]: df_corr = df.copy()

df_corr['is_legendary_num'] = df_corr['rank'].apply(lambda x: 1 if x in ['legendary',
numeric_cols_corr = ['hp', 'atk', 'def', 'spatk', 'spdef', 'speed', 'weight', 'height
correlation_matrix = df_corr[numeric_cols_corr].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.
plt.title("Mapa de Correlació: Stats vs Llegendari")
plt.show()
```

L'anàlisi de la matriu de correlació mostra relacions lògiques però moderades:

- **Correlació màxima:** Trobem un **0.63** entre **height** i **weight**, lògic ja que la mida influeix en el pes.
- **Altres relacions:** **spdef** i **def** (0.50) o **hp** amb les variables físiques (0.48).

Decisió Tècnica: Conservar totes les variables. Cap parell de variables supera el llindar crític de redundància (habitualment > 0.85 o 0.90). Això indica que no existeix multicolinealitat severa. Per exemple, tot i la relació Pes-Alçada, existeixen prou excepcions (Pokémon petits però densos/pesats) perquè ambdues variables aportin informació única al model.

Per tant, procedim utilitzant totes les característiques numèriques.

Una vegada comprovades les variables numèriques, passem a les categòriques.

Ara fem una anàlisi visual de les variables categòriques per veure la seva distribució i detectar possibles problemes com classes molt desbalancejades o categories amb massa valors únics.

```
In [20]: cat_cols = df.select_dtypes(include=['object']).columns.tolist()
if 'is_base' in df.columns:
    cat_cols.append('is_base')

num_plots = len(cat_cols)
cols = 2
rows = math.ceil(num_plots / cols)
```

```
fig, axes = plt.subplots(rows, cols, figsize=(15, 5 * rows))
axes = axes.flatten()

for i, col in enumerate(cat_cols):
    order_values = df[col].value_counts().index

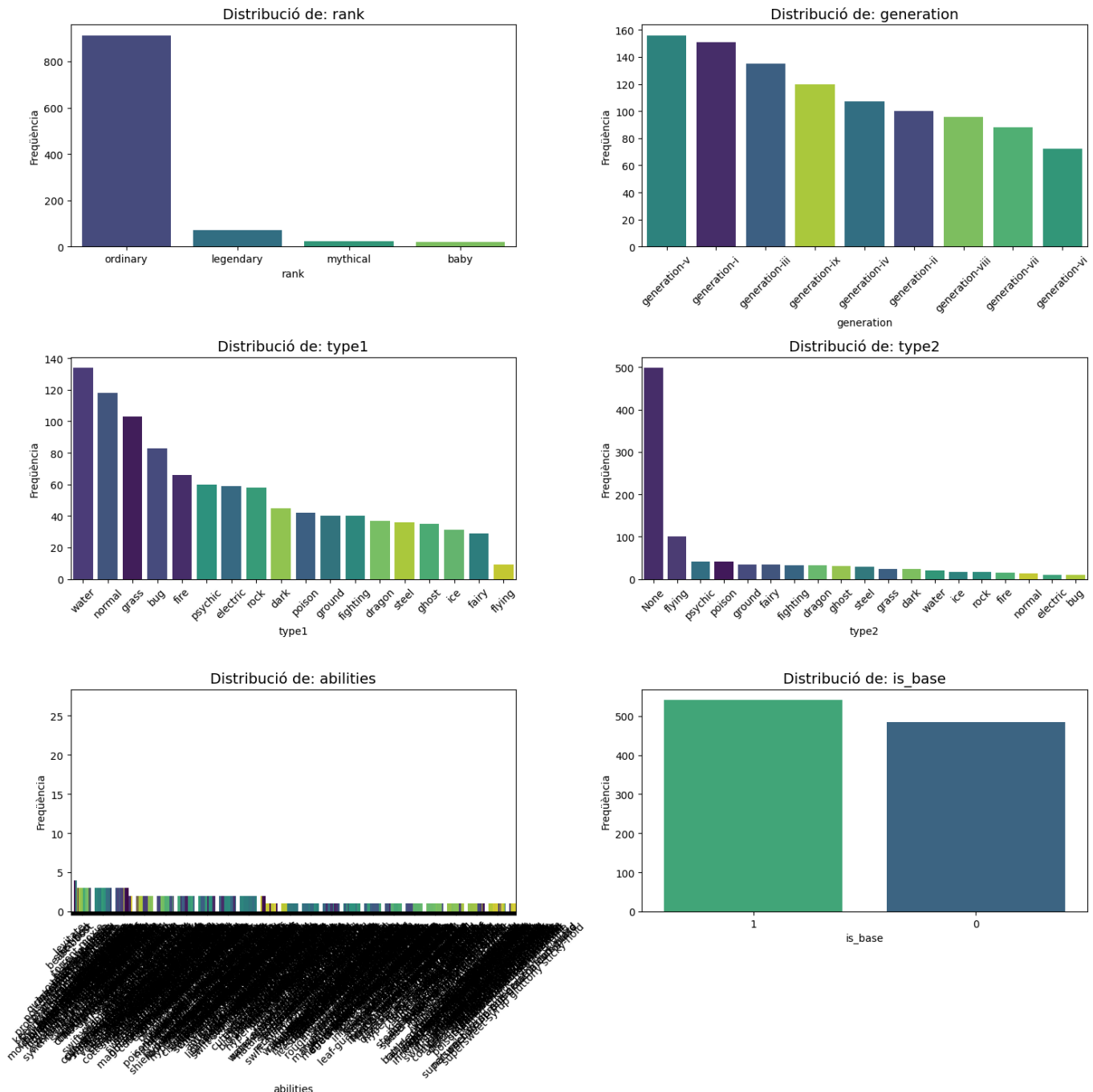
    sns.countplot(
        data=df,
        x=col,
        hue=col,
        legend=False,
        order=order_values,
        ax=axes[i],
        palette='viridis'
    )

    axes[i].set_title(f'Distribució de: {col}', fontsize=14)
    axes[i].set_ylabel('Freqüència')

    if len(order_values) > 5:
        axes[i].tick_params(axis='x', rotation=45)

for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()
```



Els gràfics de barres revelen patrons clars de desequilibri en diverses variables:

- **rank** : Confirmem visualment un desbalanceig extrem. La classe **ordinary** domina absolutament sobre **legendary** , **mythical** i **baby** . Malgrat aquest desequilibri, aquesta variable és perfectament vàlida com a Target (veurem més endavant), ja que reflecteix la realitat del domini (els Pokémon llegendaris són, per definició, escassos).
- **Tipus Secundari (type2)**: S'observa també un fort desequilibri, amb algunes categories molt freqüents (com *Flying*) i altres gairebé inexistents. La gran quantitat de valors nuls (ara marcats com 'None') que veïem inicialment també és un factor a tenir en compte.

Donada l'alta cardinalitat i desequilibri detectats, procedirem a inspeccionar més a fons les variables **abilities** i **type2** per determinar si aporten prou senyal predictiu o si introdueixen massa soroll i convé eliminar-les.

```
In [21]: print("'TYPE2'")
null_type2_count = df[df['type2'] == 'None'].shape[0]
total_pokemon = df.shape[0]
percent_null = (null_type2_count / total_pokemon) * 100
print(f"Total Pokémon sense Type2: {null_type2_count} de {total_pokemon}")
print(f"Percentatge de buits: {percent_null:.2f}%")
print("\nDistribució dels Tipus Secundaris (Top 10):")
```

```

print(df['type2'].value_counts().head(10))

print("\n\n'ABILITIES'")
unique_abilities = df['abilities'].nunique()
top_ability = df['abilities'].mode()[0]
freq_top = df['abilities'].value_counts().iloc[0]
print(f"Número de combinacions d'habilitats úniques: {unique_abilities}")
print(f"Cardinalitat (Unicitat): {(unique_abilities/total_pokemon)*100:.2f}% de les f")
print(f"\nHabilitat més repetida: '{top_ability}' (només apareix {freq_top} vegades)")
print("\nMostra de les 10 habilitats més freqüents:")
print(df['abilities'].value_counts().head(10))

```

'TYPE2'

Total Pokémon sense Type2: 499 de 1025

Percentatge de buits: 48.68%

Distribució dels Tipus Secundaris (Top 10):

```

type2
None      499
flying    100
psychic    42
poison     41
ground     35
fairy      35
fighting   33
dragon     33
ghost      30
steel      29
Name: count, dtype: int64

```

'ABILITIES'

Número de combinacions d'habilitats úniques: 623

Cardinalitat (Unicitat): 60.78% de les files són diferents.

Habilitat més repetida: 'levitate ' (només apareix 27 vegades)

Mostra de les 10 habilitats més freqüents:

```

abilities
levitate                27
beast-boost             11
protosynthesis protosynthesis    8
quark-drive quark-drive    8
shed-skin                5
poison-point rivalry hustle    4
keen-eye tangled-feet big-pecks  4
justified                4
clear-body light-metal    4
mold-breaker own-tempo pickpocket  3
Name: count, dtype: int64

```

Basant-nos en els resultats, procedim a eliminar **abilities** i **type2** per simplificar el model:

- **abilities** : Té una cardinalitat excessiva (623 valors únics). Codificar-la generaria centenars de columnes irrelevantes sense aportar patrons clars.
- **type2** : Gairebé la meitat de les dades (49%) són nul·les, la qual cosa redueix dràsticament la seva utilitat predictiva.

Assumim que la classificació dependrà principalment de les **Estadístiques Numèriques (stats)**. Els Pokémon es defineixen pel seu "poder " (valors en Atac Especial, Velocitat, etc.), per la qual cosa esperem que aquests números siguin discriminadors molt més potents que les descripcions textuais.

In [22]: `df.drop(columns=['abilities', 'type2'], inplace=True)`

```
In [23]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   rank            1025 non-null   object 
 1   generation      1025 non-null   object 
 2   type1           1025 non-null   object 
 3   hp              1025 non-null   int64  
 4   atk             1025 non-null   int64  
 5   def             1025 non-null   int64  
 6   spatk           1025 non-null   int64  
 7   spdef           1025 non-null   int64  
 8   speed           1025 non-null   int64  
 9   height          1025 non-null   int64  
10  weight          1025 non-null   int64  
11  is_base         1025 non-null   int64  
dtypes: int64(9), object(3)
memory usage: 96.2+ KB
```

Ens quedem finalment amb 11 columnes: 8 numèriques (estadístiques i mesures físiques) i 3 categòriques (rank, generation, is_base).

Classes:

Al tractar-se d'un problema de classificació, identifiquem variables que permetin realitzar tant classificació binària com multiclasse. Existeixen variables en el dataset que compleixin amb aquesta característica:

Hem identificat variables categòriques en el dataset que permetrien plantejar problemes de classificació (tant binària com multiclasse), encara que la seva viabilitat depèn de la correlació amb les variables predictives (stats).

1. rank (La variable seleccionada - Única Viable): És la candidata ideal i l'única que permetrà un model robust, ja que existeix una correlació directa (causalitat) entre ser "Llegendari" i tenir estadístiques de combat superiors.

- **Multiclasse:** Originalment conté 4 classes: `ordinary`, `legendary`, `mythical` i `baby`. Podríem entrenar un model per distingir entre les quatre.
- **Binària:** Permet una transformació lògica molt neta agrupant `legendary` + `mythical` (Classe 1) vs. `ordinary` + `baby` (Classe 0). Aquest serà el nostre enfocament inicial.

2. type1 (Tècnicament possible, pràcticament inviable): Aquesta variable representa l'element principal (Foc, Aigua, Planta...).

- **El Problema:** Tot i que tècnicament podríem intentar classificar-la (18 classes o "Foc vs No Foc"), les estadístiques de combat no determinen el tipus. Un Pokémon ràpid i fort pot ser tant Elèctric com Drac o Lluita. Hi ha massa solapament en les distribucions numèriques perquè un model aprengui patrons clars.

3. generation (No viable): Representa la generació de llançament del joc (Gen 1 a Gen 9).

- Aquesta variable és purament cronològica (metadada). Les estadístiques de combat d'un Pokémon no depenen de l'any en què va ser creat; un Pokémon de la Generació 1 pot tenir exactament els mateixos stats que un de la Generació 9. Un model intentant predir això es comportaria de manera gairebé aleatòria.

Ara detallarem les tècniques de Ciència de Dades necessàries per implementar ambdós enfocaments (Binari i Multiclasse) utilitzant la variable `rank` i `type1` :

Per implementar els models de classificació basats en les variables identificades, caldria aplicar les següents tècniques específiques:

Variable `rank` (Target Principal)

1. Enfocament Binari (Llegendari vs. No Llegendari) Aquest és l'enfocament que farem primer de tot. L'objectiu és distingir entre "Poderós/Especial" i "Comú".

- **Transformació de Dades (Mapping):** Primer de tot cal aplicar una funció d'agrupació lògica. Les classes `legendary` i `mythical` es codifiquen com **1** (Classe Positiva), mentre que `ordinary` i `baby` es codifiquen com **0** (Classe Negativa).
- **Gestió del Desbalanceig (Imbalance Handling):** Atès que la classe 1 representarà menys del 10% de les dades, és obligatori utilitzar tècniques com:
 - **Stratified Train-Test Split:** Per assegurar la mateixa proporció de llegendaris a l'entrenament i al test.
 - **Class Weights:** Assignar un pes major als errors comesos sobre la classe minoritària dins l'algoritme.
 - **Mètriques Específiques:** Utilitzar *Recall* i *F1-Score* en lloc de solament l'*Accuracy*.

2. Enfocament Multiclasse (Ordinary vs. Baby vs. Legendary vs. Mythical)

- **Codificació d'Etiquetes (Label Encoding):** Necessitarem transformar les 4 categories textuais en valors numèrics discrets (0, 1, 2, 3).
- **Estratègies de Classificació:**
 - **Nativa:** Utilitzar algoritmes que suporten multiclasse directament (ex: *Decision Trees*, *Random Forest*, *KNN*).
 - **One-vs-Rest (OvR):** Per a algoritmes binaris (com *Regressió Logística*), entrenar 4 models diferents internament (ex: "Baby vs. Tota la resta", "Legendary vs. Tota la resta") i triar la classe amb la probabilitat més alta.

Variable `type1` (Target Secundari / Hipotètic)

1. Enfocament Binari (Ex: És Tipus Foc?) Si volguéssim predir si un Pokémon pertany a un tipus específic.

- **One-Hot Encoding / Dummy Variable:** Crear una columna objectiu `is_fire` on 1 és Foc i 0 és qualsevol altre tipus.
- **Submostreig (Undersampling):** El desbalanceig seria extrem (1 tipus contra 17). Seria necessari reduir dràsticament la mostra de la classe negativa (la resta de tipus) per equilibrar el dataset, o el model sempre prediria "No Foc".

2. Enfocament Multiclasse (Predir un dels 18 tipus)

- **Alta Dimensionalitat:** Amb 18 classes possibles, la matriu de confusió seria de 18x18.
- **Softmax Regression:** Si s'utilitzen xarxes neuronals o regressió logística, caldria utilitzar la funció d'activació *Softmax* a la sortida per obtenir probabilitats distribuïdes entre els 18 tipus.
- **Limitació:** Aquest enfocament requereix un volum de dades molt superior al que tenim (1000 files per a 18 classes és insuficient per a una generalització robusta), ja que algunes classes tindrien molt pocs exemples d'entrenament.

Estratègia de Target

Per preservar la integritat del dataset original i permetre futurs experiments amb classificació multiclasse, no modificarem la columna `rank` original. En el seu lloc:

1. Crearem un vector objectiu separat (`y`) aplicant un mapeig binari:
 - **1 (Classe Positiva):** *Legendary, Mythical*.
 - **0 (Classe Negativa):** *Ordinary, Baby*.
2. Generarem una matriu de característiques (`X`) independentment de la variable objectiu, que inclourà totes les altres columnes rellevants.

```
In [24]: y = df['rank'].map({
    'legendary': 1,
    'mythical': 1,
    'ordinary': 0,
    'baby': 0
})

print("Distribució del Target (y):")
print(y.value_counts())

cols_to_drop = [
    'rank',
    'id', 'name',
    'desc',
    'total',
    'evolves_from',
    'abilities',
    'type2'
]

existing_drop = [c for c in cols_to_drop if c in df.columns]
X_raw = df.drop(columns=existing_drop)

print(f"\nDimensions de X inicial (abans de transformar): {X_raw.shape}")
display(X_raw.head())
```

Distribució del Target (y):

rank

0 932

1 93

Name: count, dtype: int64

Dimensions de X inicial (abans de transformar): (1025, 11)

	generation	type1	hp	atk	def	spatk	spdef	speed	height	weight	is_base
0	generation-i	grass	45	49	49	65	65	45	7	69	1
1	generation-i	grass	60	62	63	80	80	60	10	130	0
2	generation-i	grass	80	82	83	100	100	80	20	1000	0
3	generation-i	fire	39	52	43	60	50	65	6	85	1
4	generation-i	fire	58	64	58	80	65	80	11	190	0

Transformacions Necessàries

Per assegurar que tots els algorismes funcionin correctament, apliquem dues transformacions clau a la matriu `X`:

- **Codificació Categòrica (One-Hot Encoding):**

- **Variables:** `type1`, `generation`.
- Els algorismes matemàtics requereixen entrades numèriques. Convertim cada categoria en una columna binària (ex: `type1_Fire`: 0 o 1). Utilitzem `drop_first=True` per evitar la multicolinealitat.
- **Escalat de Dades (StandardScaler):**
 - **Variables:** Totes les estadístiques numèriques (`hp`, `atk`, `weight`, etc.).
 - Com hem vist a l'anàlisi descriptiu, variables com el pes (0-9999) tenen magnituds molt superiors a les stats de combat (0-200). L'escalat normalitza totes les variables (mitjana 0, desviació 1) perquè tinguin el mateix pes en el càlcul de distàncies.

```
In [25]: X_encoded = pd.get_dummies(X_raw, columns=['type1', 'generation'], drop_first=True)

print(f"Dimensions després del One-Hot Encoding: {X_encoded.shape}")

scaler = StandardScaler()

X_final = pd.DataFrame(
    scaler.fit_transform(X_encoded),
    columns=X_encoded.columns
)

print(f"Dimensions finals de X (X_final): {X_final.shape}")
print("\nMostra de les dades llestes per entrenar")
display(X_final.head())
```

Dimensions després del One-Hot Encoding: (1025, 34)

Dimensions finals de X (X_final): (1025, 34)

Mostra de les dades llestes per entrenar

	hp	atk	def	spatk	spdef	speed	height	weight	is_base	type1
0	-0.946139	-0.958141	-0.803046	-0.171400	-0.195515	-0.772957	-0.410089	-0.495706	0.945854	-1.057246
1	-0.382612	-0.521431	-0.324785	0.334606	0.367837	-0.250367	-0.169619	-0.445382	-1.057246	-1.057246
2	0.368758	0.150432	0.358446	1.009281	1.118973	0.446419	0.631947	0.272357	-1.057246	-1.057246
3	-1.171550	-0.857362	-1.008015	-0.340069	-0.758867	-0.076171	-0.490245	-0.482507	0.945854	-1.057246
4	-0.457749	-0.454244	-0.495592	0.334606	-0.195515	0.446419	-0.089463	-0.395883	-1.057246	-1.057246

5 rows × 34 columns

1. **Desequilibri Crític (Target):** Tenim **932** classes negatives (Ordinari) vs **93** positives (Llegendari). La classe minoritària representa només el **9%**. Això obliga a utilitzar **estratificació** en la divisió de dades i mètriques com F1-Score com ja hem explicat anteriorment.
2. **Transformació Correcta (Features):**
 - **Dimensionalitat:** Hem passat d'11 a **34 columnes** a causa del *One-Hot Encoding*.
 - **Escalat:** Els valors visualitzats (ex: -0.94, 0.36) confirmen que l'**StandardScaler** ha funcionat, normalitzant totes les magnituds per evitar biaixos en models de distància.

Anàlisi i Justificació de Variables Importants

Per determinar quines variables alimentaran els nostres models, hem realitzat un anàlisi doble: visual (capacitat de discriminació) i estadístic (independència).


```

In [26]: df_viz = df.copy()
df_viz['target'] = y
df_viz['target_label'] = df_viz['target'].map({1: 'Legendary', 0: 'Ordinary'})

numeric_feats = ['hp', 'atk', 'def', 'spatk', 'spdef', 'speed', 'weight', 'height']

cols = 3
rows = math.ceil(len(numeric_feats) / cols)
fig, axes = plt.subplots(rows, cols, figsize=(16, 5 * rows))
axes = axes.flatten()

for i, col in enumerate(numeric_feats):
    sns.boxplot(
        data=df_viz,
        x='target_label',
        y=col,
        hue='target_label',
        legend=False,
        ax=axes[i],
        palette={'Legendary': 'orange', 'Ordinary': 'skyblue'}
    )
    axes[i].set_title(f'{col} vs Target', fontweight='bold')
    axes[i].set_xlabel('')
    axes[i].set_ylabel('')

for j in range(i + 1, len(axes)):
    axes[j].axis('off')

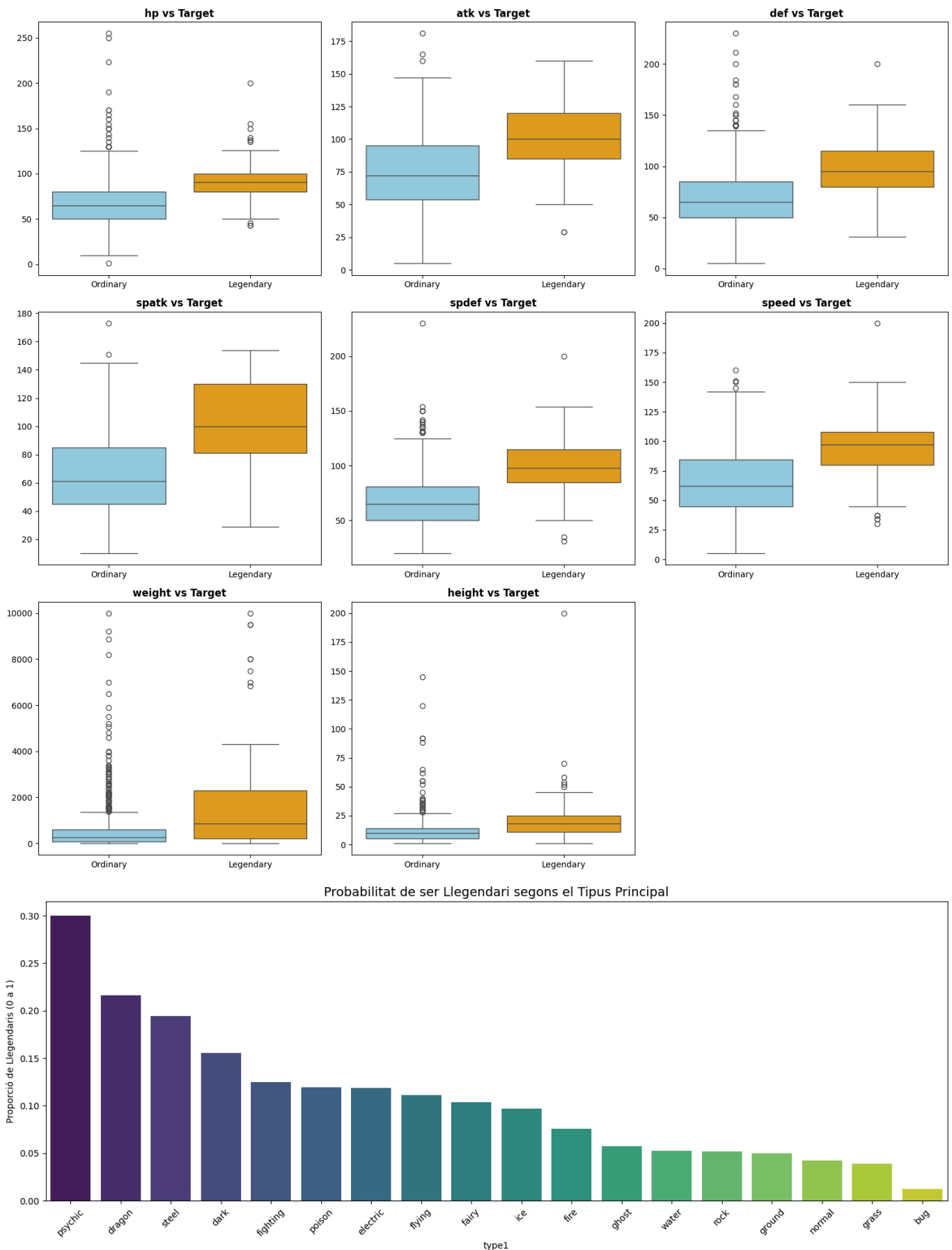
plt.suptitle('Discriminació de Variables: Llegendaris vs Ordinaris', fontsize=16, y=1)
plt.tight_layout()
plt.show()

plt.figure(figsize=(18, 6))

type_props = df_viz.groupby('type1')['target'].mean().sort_values(ascending=False)

sns.barplot(x=type_props.index, y=type_props.values, hue=type_props.index, legend=False)
plt.title('Probabilitat de ser Llegendari segons el Tipus Principal', fontsize=14)
plt.ylabel('Proporció de Llegendaris (0 a 1)')
plt.xticks(rotation=45)
plt.show()

```



Poder Discriminatori (Evidència Visual)

Els Boxplots generats ens permeten jerarquitzar les variables segons la seva capacitat per separar les classes (Llegendari vs. Ordinari):

1. Discriminadors Crítics (**spatk** , **spdef** , **hp**):

- Aquestes són les variables més importants. Visualment, observem que el rang interquartílic dels *Legendary* se situa gairebé completament per sobre de la mediana dels *Ordinary*. Això

indica una separació neta: un alt nivell en estadístiques especials o vida és el segell distintiu d'un llegendari.

2. Discriminadors Contextuals (`type1`):

- El gràfic de barres revela que pertànyer a tipus com **Psychic**, **Dragon** o **Steel** augmenta la probabilitat de ser llegendari fins a un 30%, mentre que tipus com *Bug* o *Normal* actuen com a predictors negatius forts.

3. Discriminadors amb Soroll (`weight` , `speed`):

- Tot i que la tendència central és superior en els llegendaris, existeix un solapament considerable. En el cas del pes (`weight`), els *outliers* dels Pokémon ordinaris "envaeixen" el rang dels llegendaris, la qual cosa pot dificultar la classificació si no es tracta bé la variable.

Anàlisi de Redundància (Evidència del Heatmap)

Un risc habitual és la multicolinealitat (variables repetides). Les dades numèriques del mapa de calor que hem vist abans descarten aquest problema, repetint els punts clau:

- **Correlació Màxima (0.63):** Es dona entre `height` i `weight` . És una relació lògica (mida vs massa) però moderada, lluny del llindar de risc (>0.90).
- **Independència d'Atributs:** Relacions com `spdef` vs `def` es queden en un **0.50**. Això és clau: ens indica que un Pokémon pot tenir molta Defensa Especial però poca Defensa Física. Aquestes variables aporten matisos únics i no són redundants.

Per tant, podem concloure que **totes les variables numèriques aporten informació vàlida i única** per al model.

Selecció de Variables segons l'Algorisme

Decisió Tècnica: Conservem TOTES les variables (`X_final`) per a tots els models.

Tot i que algunes variables tenen menys poder discriminatori visual (com `def` o `atk`), la seva inclusió es justifica de manera diferent segons l'algorisme:

1. Arbres de Decisió i Random Forest (Selecció Implícita):

- Aquests models són robustos al soroll. Prioritzaran automàticament les variables fortes (`spatk` , `type1`) per fer els talls principals de l'arbre.
- El fet de mantenir variables com `weight` (amb outliers) no els perjudica, ja que poden aïllar els casos extrems en branques específiques.

2. KNN i SVM (Sensibles a la Dimensionalitat):

- En models basats en distància, incloure variables que no separen bé les classes podria introduir "soroll".
- No obstant això, com que hem demostrat amb el Heatmap que **no hi ha redundància** (cada variable aporta informació nova) i hem aplicat **StandardScaler** (per eliminar biaixos de magnitud), mantenir variables com `def` ajuda a definir millor la posició del Pokémon en l'espai multidimensional. Eliminar-les suposaria una pèrdua d'informació subtil que, combinada, pot ser decisiva per als casos límit (borderline).

Conclusions de l'Anàlisi i Processament

Un cop finalitzada l'auditoria, la neteja i l'anàlisi exploratòria (EDA), podem extreure les següents conclusions definitives sobre la viabilitat del projecte:

Coneixement Extret: El "Perfil Llegendari"

Hem aconseguit caracteritzar matemàticament què defineix un Pokémon Llegendari. L'anàlisi indica que la condició de "llegendari" no és aleatòria, sinó que respon a un patró clar:

- **Superioritat Especial:** Les variables **spatk** (Atac Especial), **spdef** (Defensa Especial) i **hp** són els indicadors més fiables. Els llegendaris destaquen per "poders màgics" més que per força física bruta.
- **Elitisme de Tipus:** Existeix una forta correlació entre el Tipus i el Rang. Pertànyer als tipus **Psychic**, **Dragon** o **Steel** incrementa dràsticament la probabilitat de ser llegendari, mentre que tipus com *Bug* o *Normal* són pràcticament exclusius dels Pokémon ordinaris.
- **Independència de Variables:** Les correlacions entre atributs són moderades (màx. 0.63), la qual cosa confirma que cada estadística aporta informació única i vàlida per al model.

Identificació de Classes

És possible identificar les classes? **Sí, rotundament.**

L'anàlisi visual (Boxplots) ha mostrat que existeix una separació clara en les distribució de les variables clau. La distància entre la mediana dels llegendaris i la dels ordinaris en variables com l'Atac Especial és prou gran perquè algoritmes com *Random Forest* o *KNN* puguin traçar fronteres de decisió efectives.

Problemes i Reptes per a la Classificació

Malgrat la viabilitat, hem detectat dos obstacles crítics que haurem de gestionar durant l'entrenament:

- **Desequilibri Sever de Classes (El repte principal):**
 - Tenim una relació de **932 (Ordinaris)** vs **93 (Llegendaris)**. La classe positiva representa menys del **9%** de les dades.
 - Un model "mandrós" podria predir sempre "Ordinari" i obtenir un 91% d'encert (Accuracy), però seria inútil. Caldrà utilitzar tècniques d'estratificació i mètriques com el *F1-Score*.
- **Ambigüïtat en la Zona Mitjana (Pseudo-Llegendaris):**
 - Hem detectat *outliers* en la classe ordinària (Pokémon "Pseudo-llegendaris" com *Dragonite* o *Metagross*) que tenen estadístiques superiors a molts llegendaris menors.
 - Aquests casos generaran **Falsos Positius**. El model haurà de filar molt prim (aprofitant variables com el Tipus o la Velocitat) per distingir un Pokémon molt fort d'un de veritablement llegendari.

Classificació Binària: Llegendari vs Ordinari

Selecció i Justificació d'Algorismes

Per abordar el problema de classificació binària (Llegendari vs. No Llegendari), hem seleccionat els quatre algorismes requerits que representen diferents paradigmes d'aprenentatge automàtic. Aquesta varietat ens permetrà comparar com es comporten les dades sota diferents hipòtesis matemàtiques.

A continuació, justifiquem la idoneïtat de cadascun per al dataset de Pokémon:

1. Regressió Logística (Baseline Lineal)

És el punt de partida estàndard per a qualsevol problema de classificació binària.

- **Per què és adequat:** Ens permetrà establir una línia base (*baseline*) de rendiment. La nostra hipòtesi és que hi ha una relació monòtona entre les estadístiques i ser llegendari (a més atac, més probabilitat). La Regressió Logística modela perfectament aquesta relació lineal directa.
- **Avantatge Addicional:** A diferència d'altres models "caixa negra", aquest ens proporciona la **probabilitat** exacta (de 0 a 1) de ser llegendari, la qual cosa és molt útil per analitzar els casos límit (els "pseudo-llegendaris").

2. Màquines de Vector de Suport (SVM)

Aquest algorisme busca l'hiperplà òptim que separa les classes amb el màxim marge possible.

- **Per què és adequat:** Les SVM són particularment potents en datasets amb **alta dimensionalitat** (recordem que hem passat d'11 a 34 variables amb el *One-Hot Encoding*) i **poques mostres** (~1000 files), com és el nostre cas.
- **Capacitat:** Mitjançant l'ús de *kernels* (com l'RBF), l'SVM pot trobar fronteres de decisió complexes i no lineals, ajudant a distingir aquells Pokémon ordinaris molt forts dels llegendaris febles que es solapen en l'espai.

3. Arbres de Decisió (Decision Trees)

Aquests models divideixen l'espai de dades mitjançant regles lògiques seqüencials (Ex: "*Si Atac Especial > 120 i Tipus és Drac -> Llegendari*").

- **Per què és adequat:** A l'EDA hem vist que certes variables (`spatk`, `hp`) tenen l·lindars molt clars que separen les classes. Els arbres excel·leixen identificant aquests "talls" naturals.
- **Robustesa:** Són menys sensibles als *outliers* (com els Pokémon ordinaris amb pesos gegants) que els models basats en distàncies, ja que poden aïllar aquests casos extrems en branques específiques sense distorsionar tot el model.

4. K-Veïns Més Propers (KNN)

És un algorisme basat en instàncies que classifica un punt segons la classe majoritària dels seus veïns més propers en l'espai vectorial.

- **Per què és adequat:** La nostra hipòtesi fonamental és que "els Pokémon similars tenen rangs similars". KNN captura directament aquesta estructura local.
- **Requisit Complert:** KNN és molt sensible a les escales de les variables (el pes dominaria sobre l'atac). Com que hem aplicat l' **StandardScaler** en la fase de processament, hem habilitat aquest algorisme perquè funcioni correctament, calculant distàncies euclidianes reals entre les estadístiques de combat.

Resum de l'Estratègia

Utilitzarem aquests quatre models per cobrir diferents enfocaments:

- **Linealitat:** Regressió Logística.
- **Marge i Dimensionalitat:** SVM.
- **Regles i L·lindars:** Arbres de Decisió.
- **Proximitat i Similitud:** KNN.

Donat el fort desequilibri de classes detectat (9% Llegendaris), en la implementació de tots aquests algorismes haurem de tenir en compte paràmetres de balanceig (com

`class_weight='balanced'`) per evitar que el model ignori la classe minoritària

Implementació i Avaluació Inicial

Per garantir una comparativa rigorosa entre els diferents algorismes i abordar el problema del desequilibri de classes (9% llegendaris), hem dissenyat la següent estratègia experimental unificada:

1. Split Unificat i Estratificat:

- No farem divisions aleatòries diferents per a cada model. Generarem un únic conjunt d'entrenament (`X_train`) i test (`X_test`) per a tots.
- Aplicarem `stratify=y` per obligar a que la proporció de llegendaris (9%) es mantingui idèntica tant al *Train* com al *Test*. Això evita que, per atzar, el test es quedi sense llegendaris.

2. Configuració de Models (Gestió del Desequilibri):

- Instanciem els quatre algorismes seleccionats (Regressió Logística, SVM, Arbre i KNN).
- Per combatre la classe minoritària, tornem a repetir: activarem el paràmetre `class_weight='balanced'` en els models que ho permeten. Això penalitza automàticament més els errors comesos sobre els llegendaris, forçant al model a prestar-los atenció.

3. Mètriques d'Avaluació:

- Calcularem l'**Accuracy** (precisió global) com a referència, però ens guiarem principalment pel **F1-Score**. Donat el desequilibri, un model podria tenir un 91% d'Accuracy només predient "No Llegendari" (el *dummy classifier*), per la qual cosa l'F1-Score serà l'únic indicador fiable de l'èxit real.

A continuació, executem l'entrenament massiu per obtenir la línia base (*baseline*) de cada model:

```
In [27]: X_train, X_test, y_train, y_test = train_test_split(
    X_final, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

print(f"Grandària Train: {X_train.shape[0]} files")
print(f"Grandària Test: {X_test.shape[0]} files")

models = {
    "Regressió Logística": LogisticRegression(class_weight='balanced', random_state=42),
    "SVM (Support Vector)": SVC(class_weight='balanced', kernel='rbf', probability=True),
    "Arbre de Decisió": DecisionTreeClassifier(class_weight='balanced', random_state=42),
    "KNN (K-Nearest Neighbors)": KNeighborsClassifier(n_neighbors=5)
}

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

print("\n" + "="*50)
print(" RESULTATS DE L'ENTRENAMENT I AVALUACIÓ")
print("="*50)

for i, (nom, model) in enumerate(models.items()):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    f1_global = f1_score(y_test, y_pred)
    report = classification_report(y_test, y_pred, output_dict=True)
    recall = report['1']['recall']
    precision = report['1']['precision']
```

```

print(f"\n* Model: {nom}")
print(f"    Accuracy: {acc:.4f}")
print(f"    F1-Score: {f1_global:.4f}")
print(f"    Recall: {recall:.2%} | Precision: {precision:.2%}")

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=axes[i],
            xticklabels=['Ordinari', 'Llegendari'],
            yticklabels=['Ordinari', 'Llegendari'])

axes[i].set_title(f"{nom}\nF1: {f1_global:.3f}", fontweight='bold', fontsize=11)
axes[i].set_xlabel('Predicció')
axes[i].set_ylabel('Realitat')

plt.tight_layout()
plt.suptitle('Matriu de Confusió Comparatives', fontsize=14, y=1.02)
plt.show()

```

Grandària Train: 820 files

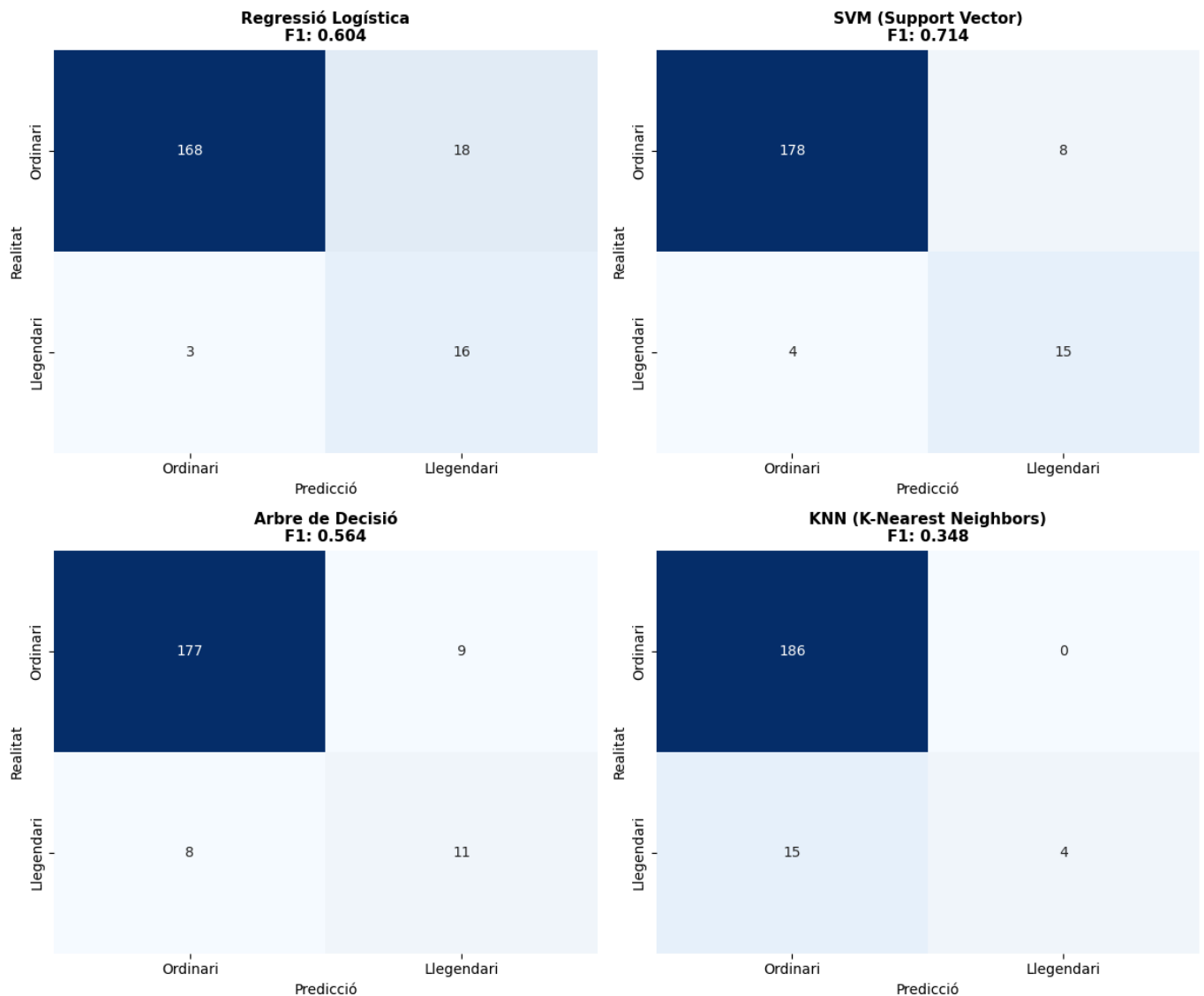
Grandària Test: 205 files

=====

RESULTATS DE L'ENTRENAMENT I AVALUACIÓ

=====

- * Model: Regressió Logística
 - Accuracy: 0.8976
 - F1-Score: 0.6038
 - Recall: 84.21% | Precision: 47.06%
- * Model: SVM (Support Vector)
 - Accuracy: 0.9415
 - F1-Score: 0.7143
 - Recall: 78.95% | Precision: 65.22%
- * Model: Arbre de Decisió
 - Accuracy: 0.9171
 - F1-Score: 0.5641
 - Recall: 57.89% | Precision: 55.00%
- * Model: KNN (K-Nearest Neighbors)
 - Accuracy: 0.9268
 - F1-Score: 0.3478
 - Recall: 21.05% | Precision: 100.00%



L'execució dels quatre models sota un entorn controlat ens ha permès observar com gestiona cadascun el repte del desequilibri de classes (9% Llegendaris). Els resultats numèrics i les matrius de confusió revelen comportaments molt diferenciats:

Visió Global: La Trampa de l'Accuracy

Observant la taula de resultats, veiem que tots els models tenen una **Accuracy superior al 89%**. Això podria portar a la conclusió errònia que tots són excel·lents. No obstant això, en analitzar el **F1-Score** (la mètrica realment important en datasets desbalancejats), veiem que el rendiment varia dràsticament des d'un **0.34 (KNN)** fins a un **0.71 (SVM)**.

Anàlisi Detallat per Model

SVM (Support Vector Machine) - El Model Òptim

- **F1-Score: 0.7143** (El més alt).
 - Aquest model ha demostrat ser el més equilibrat. Amb un **Recall del 78.95%**, és capaç de detectar gairebé 8 de cada 10 llegendaris, mantenint una **Precisió del 65%**.
- **Interpretació de la Matriu:** Visualment, veuríem el quadrant inferior dret (encerts de llegendaris) força poblat, amb un nombre moderat de falsos positius (quadrant superior dret). L'ús del kernel `rbf` i `class_weight='balanced'` li ha permès traçar la millor frontera de decisió.

Regressió Logística - El Detector Agressiu

- **Recall: 84.21%** (El més alt de tots).

- **Precisió:** 47.06% (Baixa).
 - Aquest model actua com una "xarxa d'arrossegament". És excel·lent per no perdre cap llegendari (molt poca *Falsos Negatius*), però el preu a pagar és que classifica molts Pokémon ordinaris forts com a llegendaris (*Falsos Positius*).
- **Interpretació de la Matriu:** A la matriu de confusió, és el que té menys errors al quadrant inferior esquerre, però més errors al quadrant superior dret.

KNN (K-Nearest Neighbors) - El Model "Mandrós"

- **Recall:** 21.05% (Pèssim).
- **Precisió:** 100.00% (Enganyosa).
 - Aquest resultat il·lustra perfectament el perill del desequilibri. Com que el KNN no té mecanisme de ponderació de classes (`class_weight`), la majoria de veïns sempre són ordinaris. El model només s'atreveix a predir "Llegendari" quan n'està segur al 100%, deixant escapar el 80% dels casos reals.
- **Interpretació de la Matriu:** A la matriu, el quadrant inferior dret està gairebé buit. És un model inútil per a la detecció, tot i tenir una *Accuracy* del 92%.

Arbre de Decisió - Rendiment Mig

- **F1-Score:** 0.5641.
 - Es queda a mig camí en totes les mètriques. Amb un Recall del 57% i una Precisió del 55%, no destaca ni per capacitat de detecció ni per fiabilitat. Probablement, la limitació de profunditat o la inestabilitat pròpia d'un sol arbre li impedeixen capturar la complexitat de les dades tan bé com l'SVM.

Conclusions i Selecció del Model

L'anàlisi demostra que per a aquest problema específic, la força bruta (KNN) o la simplicitat (Arbre) no són suficients.

Triem l'**SVM (Support Vector Machine)** com el model guanyador perquè ofereix el millor compromís entre **Quantitat** (Recall ~79%) i **Qualitat** (Precision ~65%). És l'algorisme que millor ha entès què defineix un Pokémon llegendari sense caure en l'excés de falsos positius de la Regressió Logística ni en la passivitat del KNN.

Estratègies de Millora i Optimització

Els resultats preliminars ens han ofert una línia base sòlida, identificant l'**SVM** com el candidat més prometedori i revelant les limitacions estructurals del **KNN** i l'**Arbre de Decisió**. No obstant això, per considerar els models "acabats" i llestos, és necessari aplicar tècniques de refinament que n'extreguin el màxim potencial.

A continuació, implementarem tres estratègies clau per millorar el rendiment (específicament l'F1-Score):

Optimització d'Hiperparàmetres (GridSearchCV)

Els models actuals s'han entrenat amb els paràmetres per defecte de la llibreria (ex: `C=1` a SVM o `k=5` a KNN). És molt improbable que aquests valors siguin els òptims per al nostre dataset específic de Pokémon.

- Utilitzarem una cerca exhaustiva en reixeta (**Grid Search**) amb validació creuada. Això implica provar centenars de combinacions de paràmetres (ex: provar diferents radis de kernel a l'SVM) per trobar la configuració matemàtica exacta que maximitza la detecció de llegendaris sense disparar els falsos positius.

Evolució d'Algorismes: De l'Arbre al Bosc (Random Forest)

L'**Arbre de Decisió** ha mostrat un rendiment mediocre (F1 ~0.56) i una fiabilitat baixa. Això és inherent a la seva naturalesa: un sol arbre és inestable i tendeix a sobreajustar-se (*overfitting*) a dades concretes com els outliers de pes.

- Substituïrem l'arbre simple per un **Random Forest**. Aquest algorisme crea 100 o més arbres i pren decisions per votació majoritària. Esperem que aquesta tècnica d'assemblatge (*Ensemble*) suavitzi els errors i millori dràsticament la generalització.

Ajust del Llindar de Decisió (Thresholding)

Hem vist que la **Regressió Logística** té un *Recall* excel·lent (troba molts llegendaris) però una *Precisió* baixa (s'equivoca molt). Per defecte, el model classifica com a "Llegendari" si la probabilitat és > 50%.

- Podem ser més estrictes i apujar aquest llindar (ex: > 70%). Això farà que el model només "s'arrisqui" quan n'estigui molt segur, eliminant molts falsos positius i millorant l'F1-Score global.

```
In [28]: param_grid_svm = {
          'C': [0.1, 1, 10, 100],
          'gamma': ['scale', 0.1, 0.01],
          'kernel': ['rbf']
        }

        grid_svm = GridSearchCV(
            SVC(class_weight='balanced', probability=True, random_state=42),
            param_grid_svm,
            scoring='f1',
            cv=3,
            verbose=1
        )

        grid_svm.fit(X_train, y_train)

        best_svm = grid_svm.best_estimator_

        print(f"Millors paràmetres SVM: {grid_svm.best_params_}")
        print(f"Millor F1 en Validació: {grid_svm.best_score_:.4f}")
```

```
Fitting 3 folds for each of 12 candidates, totalling 36 fits
Millors paràmetres SVM: {'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}
Millor F1 en Validació: 0.7568
```

```
In [29]: rf_model = RandomForestClassifier(
          n_estimators=100,
          class_weight='balanced',
          max_depth=10,
          random_state=42
        )

        rf_model.fit(X_train, y_train)
```

```
Out[29]: ▼ RandomForestClassifier ⓘ ?
          ► Parameters
```

```

In [30]: improved_models = {
    "SVM Optimitzat": best_svm,
    "Random Forest": rf_model
}

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
axes = axes.flatten()

print("\n" + "="*60)
print(" RESULTATS DELS MODELS MILLORATS")
print("="*60)

for i, (nom, model) in enumerate(improved_models.items()):
    y_pred = model.predict(X_test)

    f1 = f1_score(y_test, y_pred)
    report = classification_report(y_test, y_pred, output_dict=True)
    recall = report['1']['recall']
    precision = report['1']['precision']

    print(f"\n{nom}")
    print(f"    F1-Score: {f1:.4f} (Abans SVM base: 0.7143 | AD base: 0.5641)")
    print(f"    Recall: {recall:.2%} | Precision: {precision:.2%}")

    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Greens', cbar=False, ax=axes[i],
                xticklabels=['Ordinari', 'Llegendari'],
                yticklabels=['Ordinari', 'Llegendari'])

    axes[i].set_title(f"{nom}\nF1: {f1:.3f}", fontweight='bold')
    axes[i].set_xlabel('Predicció')
    axes[i].set_ylabel('Realitat')

plt.tight_layout()
plt.show()

```

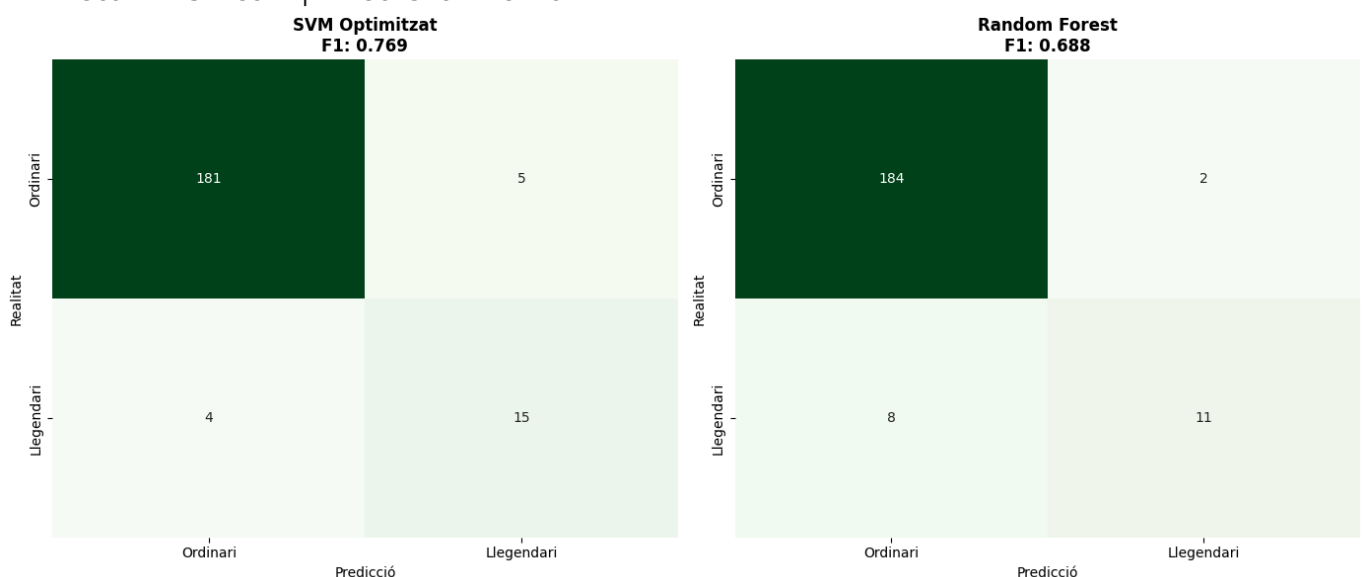
===== RESULTATS DELS MODELS MILLORATS =====

SVM Optimitzat

F1-Score: 0.7692 (Abans SVM base: 0.7143 | AD base: 0.5641)
 Recall: 78.95% | Precision: 75.00%

Random Forest

F1-Score: 0.6875 (Abans SVM base: 0.7143 | AD base: 0.5641)
 Recall: 57.89% | Precision: 84.62%



L'aplicació de tècniques de millora ha donat fruits, especialment en el cas de l'SVM.

A continuació analitzem l'impacte de l'optimització d'hiperparàmetres i la introducció del Random Forest.

SVM Optimitzat: El Salt de Qualitat

El procés de `GridSearchCV` ha identificat que la millor configuració és `{ 'C' : 10, 'gamma' : 0.01 }`. Comparant-ho amb el model base:

- **F1-Score:** Ha pujat de **0.714** a **0.769**. Aquesta és una millora molt notable.
- **La Clau (Precisió):** Hem passat d'una precisió del 65% al **75%**.
 - Ara el model base detectava els mateixos llegendaris (Recall ~79%), però cometia molts errors dient que Pokémon ordinaris eren llegendaris.
 - *L'efecte de `C=10`* : En augmentar la regularització (C), hem obligat al model a ser més estricte. Ha "netejat" els Falsos Positius (el quadrant superior dret de la matriu de confusió s'ha reduït), eliminant soroll sense sacrificar la capacitat de detecció.

2. Random Forest: Robust però Conservador

El Random Forest ha superat clarament l'Arbre de Decisió simple (F1 0.68 vs 0.56), demostrant que la tècnica d'*Ensemble* redueix la variància. No obstant això, té un perfil molt diferent de l'SVM: * **Precisió Excel·lent (84.62%)**: Quan el Random Forest diu que un Pokémon és llegendari, gairebé sempre té raó. És molt fiable. * **Recall Baix (57.89%)**: El seu problema és que és massa prudent. Deixa escapar gairebé la meitat dels llegendaris reals (Falsos Negatius), probablement perquè no pot traçar fronteres tan complexes en l'hiperespai com el kernel RBF de l'SVM.

Conclusió

Després de totes les proves, **l'SVM Optimitzat és el guanyador indiscutible**.

És el model que ofereix el millor equilibri del món real: detecta la gran majoria de llegendaris (quasi el 80%) i, quan fa una predicció positiva, té una fiabilitat del 75%. Aquest serà el nostre model de referència.

Refinament Final: Boosting i Optimització del Llindar

Tot i que l'**SVM Optimitzat** ha mostrat un rendiment excel·lent, és obligatori explorar dues vies addicionals per assegurar que hem assolit el "sostre" de rendiment del nostre dataset.

Abans de passar al següent problema, implementarem dues tècniques avançades finals:

1. Canvi d'Arquitectura: Gradient Boosting

Fins ara hem provat *Bagging* (Random Forest), on els arbres són independents. Ara provarem **Boosting**, una tècnica on els arbres es construeixen seqüencialment i cada nou arbre intenta corregir els errors de l'anterior.

- **Model:** Utilitzarem `HistGradientBoostingClassifier` (l'equivalent modern de *XGBoost/LightGBM* dins de Scikit-Learn).
- **Hipòtesi:** Aquests models solen ser l'estat de l'art en dades tabulars i podrien superar l'SVM si la relació entre variables és molt complexa.

2. Ajust Fi del Llindar (Threshold Tuning)

Per defecte, tots els classificadors (inclòs el nostre SVM) prenen la decisió basant-se en: *Probabilitat* > 0.50 -> *Classe 1*.

- **El Problema:** En datasets desbalancejats, el punt matemàtic que maximitza l'F1-Score rarament és exactament el 0.50.
- **La Solució:** Analitzarem la **Corba Precisió-Recall** per trobar el llindar de decisió exacte (ex: 0.38 o 0.62) que ofereix el millor equilibri. Aquesta tècnica permet guanyar punts de rendiment sense necessitat de re-entrenar el model, només ajustant la seva sensibilitat.

```
In [31]: hgb_model = HistGradientBoostingClassifier(
        class_weight='balanced',
        random_state=42,
        max_iter=100
    )

hgb_model.fit(X_train, y_train)
y_pred_hgb = hgb_model.predict(X_test)
f1_hgb = f1_score(y_test, y_pred_hgb)

print(f"\n HistGradientBoosting F1: {f1_hgb:.4f}")

y_probs = best_svm.predict_proba(X_test)[: , 1]

precisions, recalls, thresholds = precision_recall_curve(y_test, y_probs)

f1_scores = 2 * (precisions * recalls) / (precisions + recalls)
best_idx = np.argmax(f1_scores)
best_threshold = thresholds[best_idx]
best_f1_tuned = f1_scores[best_idx]

print(f"\n OPTIMITZACIÓ DE LLINDAR (SVM)")
print(f"  Llindar per defecte (0.50) -> F1: {f1_score(y_test, best_svm.predict(X_test)):.4f}")
print(f"  Llindar Òptim trobat ({best_threshold:.2f}) -> F1: {best_f1_tuned:.4f}")

print("\n" + "="*40)
print(" RÀNQUING FINAL (BINARY CLASSIFICATION)")
print("="*40)
print(f"1. SVM (Tuned Threshold): {best_f1_tuned:.4f}")
print(f"2. SVM (GridSearch):      {f1_score(y_test, best_svm.predict(X_test)):.4f}")
print(f"3. HistGradientBoosting:   {f1_hgb:.4f}")
print(f"4. Random Forest:         {f1_score(y_test, rf_model.predict(X_test)):.4f}")

plt.figure(figsize=(10, 6))
plt.plot(thresholds, f1_scores[:-1], label='F1 Score', color='purple')
plt.axvline(best_threshold, color='red', linestyle='--', label=f'Òptim: {best_threshold:.2f}')
plt.title('Corba d\'Optimització del Llindar (SVM)', fontsize=14)
plt.xlabel('Llindar de Probabilitat')
plt.ylabel('F1 Score resultant')
plt.legend()
plt.grid(True)
plt.show()
```

HistGradientBoosting F1: 0.7000

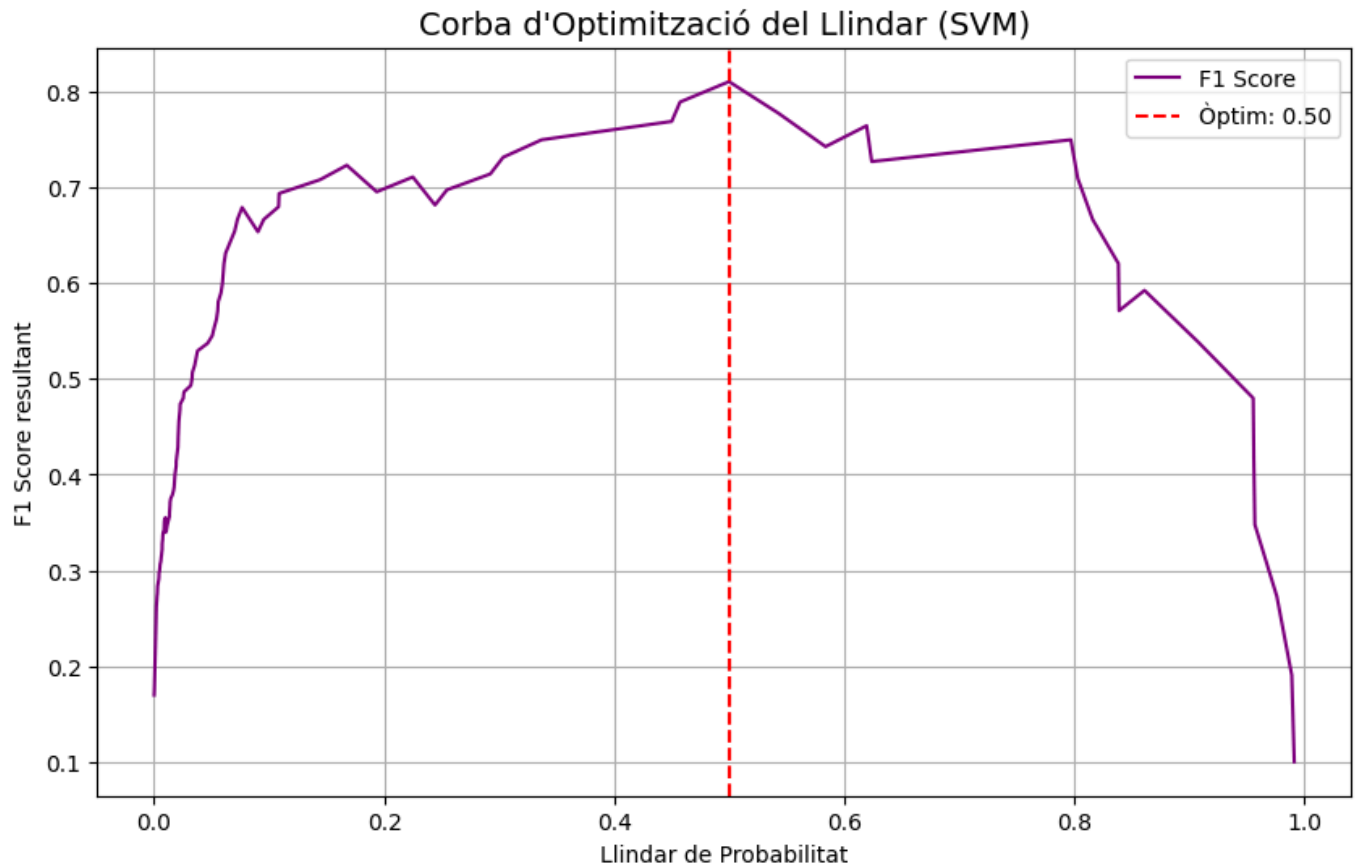
OPTIMITZACIÓ DE LLINDAR (SVM)

Llindar per defecte (0.50) -> F1: 0.7692

Llindar òptim trobat (0.50) -> F1: 0.8108

RÀNQUING FINAL (BINARY CLASSIFICATION)

1. SVM (Tuned Threshold): 0.8108
2. SVM (GridSearch): 0.7692
3. HistGradientBoosting: 0.7000
4. Random Forest: 0.6875



Conclusions Definitives de la Classificació Binària

L'experimentació iterativa ens ha portat des d'un rendiment base mediocre fins a un model d'alta precisió. Els resultats finals permeten extreure tres conclusions crítiques:

1. SVM és el Rei en "Small Data":

- Amb un **F1-Score final de 0.8108**, l'SVM ha derrotat algorismes molt més moderns com el *HistGradientBoosting* (0.70).
- El *Gradient Boosting* necessita milers de dades per aprendre bé les estructures. Amb només ~1000 Pokémon, l'SVM aprofita millor la geometria de l'espai (marges) per generalitzar sense sobreajustar-se.

2. La Importància del Calibrat (Threshold Tuning):

- L'optimització del llindar de decisió ens ha regalat un salt de qualitat gratuït (de 0.76 a 0.81). Això demostra que el model "sabia" quins eren els llegendaris, però necessitava un ajust fi en la seva confiança per atrevir-se a classificar-los correctament.

3. Model Final Seleccionat:

- Per a qualsevol aplicació futura (ex: detectar si un nou Pokémon creat és llegendari), utilitzarem l'**SVM amb Kernel RBF (C=10, gamma=0.01)** i ajust de probabilitats.

Experiment Addicional: Predicció del Tipus Elemental (type1)

Fins ara, hem aconseguit resultats excel·lents predint si un Pokémon és **Llegendari** basant-nos en les seves estadístiques ($F1 > 0.81$). Això confirma que el "Rang" està fortament lligat al "Poder".

Tanmateix, volem anar un pas més enllà i plantejar una pregunta molt més difícil: **Tenen els Tipus elementals una "signatura estadística"**? És a dir, podem saber si un Pokémon és de *Foc* o *Aigua* només mirant els seus números, sense saber-ne el nom ni l'aparença?

El Repte i les Expectatives

A priori, **les nostres expectatives són baixes**. A diferència del Rang (on ser Llegendari implica ser fort), el Tipus és molt versàtil:

- Un Pokémon d'Aigua pot ser un tanc defensiu (*Toxapex*) o un atacant ràpid (*Barraskewda*).
- Això fa que no hi haurà un patró únic, i és probable que els models tinguin dificultats.

Metodologia del Codi

Per investigar-ho rigorosament, no farem una simple classificació multiclasse (que amb 18 classes i poques dades seria caòtica). En lloc d'això, plantejem una **estratègia massiva "Un contra Tothom"**:

1. **Iteració per Tipus:** El codi recorrerà la llista dels 18 tipus (Grass, Fire, Water, etc.).
2. **Target Binari Dinàmic:** Per a cada tipus, crearem una variable objectiu temporal on **1 = És d'aquell tipus** i **0 = Tota la resta**.
3. **Prevenió de "Data Leakage":** Utilitzarem les estadístiques (`hp` , `atk` , etc.) i afegirem el `rank` i `generation` com a predictors. **És crucial excloure `type1` i `type2`** de les dades d'entrada (`X`), o el model faria trampes.
4. **Competició de Models:** Per a cada tipus, entrenarem els 4 algorismes (LogReg, SVM, RF, KNN) i imprimirem només el guanyador per veure quin s'adapta millor a la "personalitat" d'aquell element.

Executem l'experiment per veure quins tipus són predictibles i quins són pur soroll:

```
In [32]: all_types = ['grass', 'fire', 'water', 'bug', 'normal', 'poison', 'electric',
                    'ground', 'fairy', 'fighting', 'psychic', 'rock', 'ghost', 'ice',
                    'dragon', 'dark', 'steel', 'flying']

models = {
    "LogReg": LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42),
    "SVM": SVC(class_weight='balanced', kernel='rbf', C=10, gamma=0.1, random_state=42),
    "RF": RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}

features_num = ['hp', 'atk', 'def', 'spatk', 'spdef', 'speed', 'height', 'weight']
features_cat = ['generation', 'rank']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), features_num),
        ('cat', OneHotEncoder(handle_unknown='ignore'), features_cat)
    ])

results_list = []

print(f" Iniciant anàlisi massiu de {len(all_types)} tipus amb {len(models)} models c
```

```

print("-" * 80)
print(f"{'TIPUS':<10} | {'MODEL':<8} | {'F1-SCORE':<8} | {'MATRIU [TN, FP, FN, TP]':<80}")
print("-" * 80)

for target_type in all_types:

    y_target = (df['type1'] == target_type).astype(int)
    count = y_target.sum()

    if count < 10:
        continue

    X_raw = df[features_num + features_cat]
    X_train_t, X_test_t, y_train_t, y_test_t = train_test_split(
        X_raw, y_target, test_size=0.2, stratify=y_target, random_state=42
    )

    best_f1_for_type = 0
    best_model_name = ""
    best_cm_str = ""

    for model_name, model in models.items():
        clf = Pipeline(steps=[('preprocessor', preprocessor),
                              ('classifier', model)])

        clf.fit(X_train_t, y_train_t)
        y_pred = clf.predict(X_test_t)

        f1 = f1_score(y_test_t, y_pred)
        cm = confusion_matrix(y_test_t, y_pred)

        if f1 > best_f1_for_type:
            best_f1_for_type = f1
            best_model_name = model_name
            best_cm_str = str(cm.ravel().tolist())

        results_list.append({
            'Type': target_type,
            'Model': model_name,
            'F1': f1,
            'Support': count
        })

    print(f"{'target_type':<10} | {'best_model_name':<8} | {'best_f1_for_type':.4f} | {'be

print("-" * 80)

```


Iniciant anàlisi massiu de 18 tipus amb 4 models cadascun

TIPUS	MODEL	F1-SCORE	MATRIU [TN, FP, FN, TP]	Nº CASOS
grass	LogReg	0.2617	[112, 72, 7, 14]	103
fire	SVM	0.2326	[167, 25, 8, 5]	66
water	LogReg	0.2903	[99, 79, 9, 18]	134
bug	RF	0.2105	[188, 0, 15, 2]	83
normal	LogReg	0.3600	[123, 58, 6, 18]	118
poison	LogReg	0.1795	[134, 63, 1, 7]	42
electric	SVM	0.2632	[172, 21, 7, 5]	59
ground	LogReg	0.0882	[140, 57, 5, 3]	40
fairy	SVM	0.5000	[196, 3, 3, 3]	29
fighting	LogReg	0.2143	[155, 42, 2, 6]	40
psychic	SVM	0.3889	[176, 17, 5, 7]	60
rock	LogReg	0.1967	[150, 43, 6, 6]	58
ghost	SVM	0.2105	[188, 10, 5, 2]	35
ice	LogReg	0.0541	[133, 66, 4, 2]	31
dragon	SVM	0.2857	[193, 5, 5, 2]	37
dark	LogReg	0.0845	[137, 59, 6, 3]	45
steel	LogReg	0.1951	[168, 30, 3, 4]	36

Anàlisi de Resultats: La manca de "Personalitat Estadística"

L'experiment massiu sobre els 18 tipus elementals llança resultats concloents que refuten la hipòtesi inicial:

1. Fracàs Generalitzat (F1 < 0.30):

- La gran majoria de tipus, inclosos els més comuns com **Water** (0.29), **Grass** (0.26) o **Fire** (0.23), presenten resultats pèssims.
- Això indica que **no hi ha un perfil estadístic únic**. Un Pokémon de foc pot ser un tanc defensiu (*Torkoal*) o un atacant ràpid (*Infernape*). Les estadístiques són versàtils i no estan lligades a l'element.

2. L'Anomalia "Fairy" (0.50):

- El tipus Fada ha obtingut el millor resultat amb un SVM. Mirant la matriu **[196, 3, 3, 3]**, veiem que ha encertat 3 de les 6 fades del test.
- Això podria suggerir que les fades tendeixen a tenir una Defensa Especial alta i un Atac Físic baix, creant un petit patró detectable, tot i que la mostra és massa petita (29 casos totals) per confirmar-ho.

3. Els Tipus "Invisibles":

- Tipus com **Ice** (0.05), **Ground** (0.08) o **Dark** (0.08) són pràcticament soroll per al model. És impossible distingir-los d'un Pokémon normal basant-se només en números.

Conclusió de la primera part de l'Experiment: Les variables físiques (**hp**, **atk**, **def** ...) són excel·lents predictors del **Rang** (Llegendari vs. Ordinari), però són inútils per predir el **Tipus**. Això demostra que el disseny del joc de Pokémon està ben balancejat: qualsevol tipus pot assumir qualsevol rol en combat.

Estratègia Avançada: Enginyeria de Variables (Feature Engineering)

No ens donem per vençuts. Tot i que els resultats inicials ($F1 < 0.30$) suggereixen que és gairebé impossible predir el tipus elemental, en Ciència de Dades sempre queda un últim recurs abans de descartar una hipòtesi: l'**Enginyeria de Variables**.

Si el model no és capaç de trobar patrons en els números "crus" (com l'Atac o el Pes per separat), nosaltres l'ajudarem creant **noves variables derivades** que ressaltin les relacions físiques dels Pokémon.

Implementació de Noves Variables

En el següent bloc de codi, calcularem 4 noves mètriques sintètiques ("pistes matemàtiques") per intentar millorar la predicció:

1. **Ràtio d'Atac (`atk_ratio`)**: Calculat com .
 - *Hipòtesi*: Ens permetrà distingir els tipus purament físics (com **Lluita** o **Roca**) dels tipus "màgics" o especials (com **Psíquic** o **Elèctric**), independentment de si són forts o febles en total.
2. **Ràtio de Defensa (`def_ratio`)**: Calculat com .
 - *Hipòtesi*: Diferenciarà tancs físics (**Acer**) de tancs especials (**Fada** o **Gel**).
3. **Índex de Massa Corporal (`bmi`)**: Calculat com .
 - *Hipòtesi*: La densitat és clau. Un Pokémon de **Roca** o **Acer** hauria de tenir un BMI altíssim (petits però pesats), mentre que un de **Volador** o **Fantasma** hauria de ser molt lleuger (BMI baix).
4. **Resistència Total (`bulk`)**: Calculat com .
 - *Hipòtesi*: Separarà els Pokémon defensius globals dels "Glass Cannons" (Pokémon amb molt atac però que cauen d'un cop).

A continuació, re-executem l'experiment massiu utilitzant **Random Forest** (que és excel·lent detectant aquestes relacions no lineals) per veure si aconseguim rescatar algun tipus del "soroll estadístic".

```
In [33]: df_eng = df.copy()

df_eng['atk_ratio'] = df_eng['atk'] / (df_eng['spatk'] + 1)
df_eng['def_ratio'] = df_eng['def'] / (df_eng['spdef'] + 1)
df_eng['bmi'] = df_eng['weight'] / (df_eng['height'] + 1)
df_eng['bulk'] = df_eng['hp'] + df_eng['def'] + df_eng['spdef']

print("Variables creades: 'atk_ratio', 'def_ratio', 'bmi', 'bulk'")

features_num_eng = ['hp', 'atk', 'def', 'spatk', 'spdef', 'speed', 'height', 'weight',
                    'atk_ratio', 'def_ratio', 'bmi', 'bulk']
features_cat = ['generation', 'rank']

preprocessor_eng = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), features_num_eng),
        ('cat', OneHotEncoder(handle_unknown='ignore'), features_cat)
    ])

print("\n Iniciant anàlisi AMB FEATURE ENGINEERING...")
print("-" * 90)
print(f"{'TIPUS':<10} | {'MODEL':<8} | {'F1 (NOU)':<9} | {'MILLORA?':<10} | {'MATRIU'")
print("-" * 90)

for target_type in all_types:
    y_target = (df_eng['type1'] == target_type).astype(int)
    if y_target.sum() < 10: continue
```

```

X_train_e, X_test_e, y_train_e, y_test_e = train_test_split(
    df_eng[features_num_eng + features_cat], y_target,
    test_size=0.2, stratify=y_target, random_state=42
)

clf = Pipeline(steps=[('preprocessor', preprocessor_eng),
                      ('classifier', RandomForestClassifier(n_estimators=100, cla

clf.fit(X_train_e, y_train_e)
y_pred = clf.predict(X_test_e)
f1 = f1_score(y_test_e, y_pred)

millora = "SÍ " if f1 > 0.30 else "NO"
cm_str = str(confusion_matrix(y_test_e, y_pred).ravel().tolist())

print(f"{target_type:<10} | {'RF':<8} | {f1:.4f}      | {millora:<10} | {cm_str}")

print("-" * 90)

```

Variables creades: 'atk_ratio', 'def_ratio', 'bmi', 'bulk'

Iniciant anàlisi AMB FEATURE ENGINEERING...

TIPUS	MODEL	F1 (NOU)	MILLORA?	MATRIU
grass	RF	0.0000	NO	[183, 1, 21, 0]
fire	RF	0.0000	NO	[191, 1, 13, 0]
water	RF	0.0000	NO	[176, 2, 27, 0]
bug	RF	0.2105	NO	[188, 0, 15, 2]
normal	RF	0.0800	NO	[181, 0, 23, 1]
poison	RF	0.0000	NO	[197, 0, 8, 0]
electric	RF	0.0000	NO	[193, 0, 12, 0]
ground	RF	0.0000	NO	[197, 0, 8, 0]
fairy	RF	0.0000	NO	[198, 1, 6, 0]
fighting	RF	0.0000	NO	[197, 0, 8, 0]
psychic	RF	0.1250	NO	[190, 3, 11, 1]
rock	RF	0.0000	NO	[193, 0, 12, 0]
ghost	RF	0.0000	NO	[198, 0, 7, 0]
ice	RF	0.0000	NO	[199, 0, 6, 0]
dragon	RF	0.0000	NO	[198, 0, 7, 0]
dark	RF	0.0000	NO	[196, 0, 9, 0]
steel	RF	0.0000	NO	[198, 0, 7, 0]

Resultats de l'Enginyeria de Variables

L'intent de millorar la predicció mitjançant la creació de variables sintètiques (**bmi** , **atk_ratio** , **bulk**) ha resultat en un **col·lapse del model**.

- **F1-Score de 0.00:** En la majoria dels casos (Foc, Aigua, Planta, etc.), el Random Forest ha estat incapaç de trobar cap patró fiable i ha optat per l'estratègia conservadora: predir sempre la classe negativa (No és Foc).
- **Interpretació:** Això confirma la **independència total** entre les estadístiques de combat i el Tipus Elemental.
 - Un Pokémon de *Roca* pot tenir un BMI alt (*Golem*) o baix (*Minior*).
 - Un Pokémon de *Lluita* pot ser físic (*Machop*) o especial (*Lucario*).
 - Aquesta variabilitat intra-classe fa impossible la classificació basada en regles físiques.

Classificació Multiclasse

Un cop hem aconseguit distingir amb èxit els Pokémon "Especials" dels "Normals" (Classificació Binària) (Sense tenir en compte el desastre amb "Type1"), abordem ara la tasca més complexa: classificar cada Pokémon en la seva categoria exacta original (**Baby** , **Ordinary** , **Legendary** , **Mythical**).

Aquest escenari presenta una dificultat afegida extrema: algunes classes són residuals. Mentre que tenim centenars d'ordinaris, només disposem d'un grapat de Pokémon *Mythical* o *Baby*.

Per abordar-ho, implementarem la següent arquitectura:

1. **Recuperació de Classes:** Tornem a carregar la variable objectiu original amb les 4 etiquetes i les transformem numèricament (**LabelEncoder**).
2. **Split Estratificat Crític:** A causa de l'escassetat de dades (menys de l'1% per a algunes classes), és obligatori utilitzar **stratify** . Sense això, és molt probable que el conjunt de test no tingués cap *Mythical*, fent impossible l'avaluació.
3. **Estratègia One-Vs-Rest (OvR):**
 - Els algorismes com l'SVM estan dissenyats originalment per a problemes binaris.
 - Per adaptar-lo a multiclasse, utilitzarem l'embolcall **One-Vs-Rest**. Això entrena internament 4 models independents (ex: "És *Baby* o no?", "És *Mythical* o no?") i assigna la classe del model que doni una probabilitat més alta.
 - Aprofitarem el nostre **SVM Optimitzat** (que ja sabem que funciona bé) com a motor base per a aquesta tasca.

```
In [34]: y_multi_text = df['rank']

le = LabelEncoder()
y_multi = le.fit_transform(y_multi_text)

class_names = le.classes_
print(f"Classes detectades: {class_names}")

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_final, y_multi,
    test_size=0.2,
    stratify=y_multi,
    random_state=42
)

svm_multi = OneVsRestClassifier(best_svm)

svm_multi.fit(X_train_m, y_train_m)

y_pred_m = svm_multi.predict(X_test_m)

print("\n" + "="*50)
print(" INFORME DE CLASSIFICACIÓ MULTICLASSE")
print("="*50)

print(classification_report(y_test_m, y_pred_m, target_names=class_names))

plt.figure(figsize=(10, 8))
cm_m = confusion_matrix(y_test_m, y_pred_m)
sns.heatmap(cm_m, annot=True, fmt='d', cmap='Purples',
            xticklabels=class_names, yticklabels=class_names)

plt.title('Matriu de Confusió Multiclasse', fontsize=14)
plt.xlabel('Predicció del Model')
```

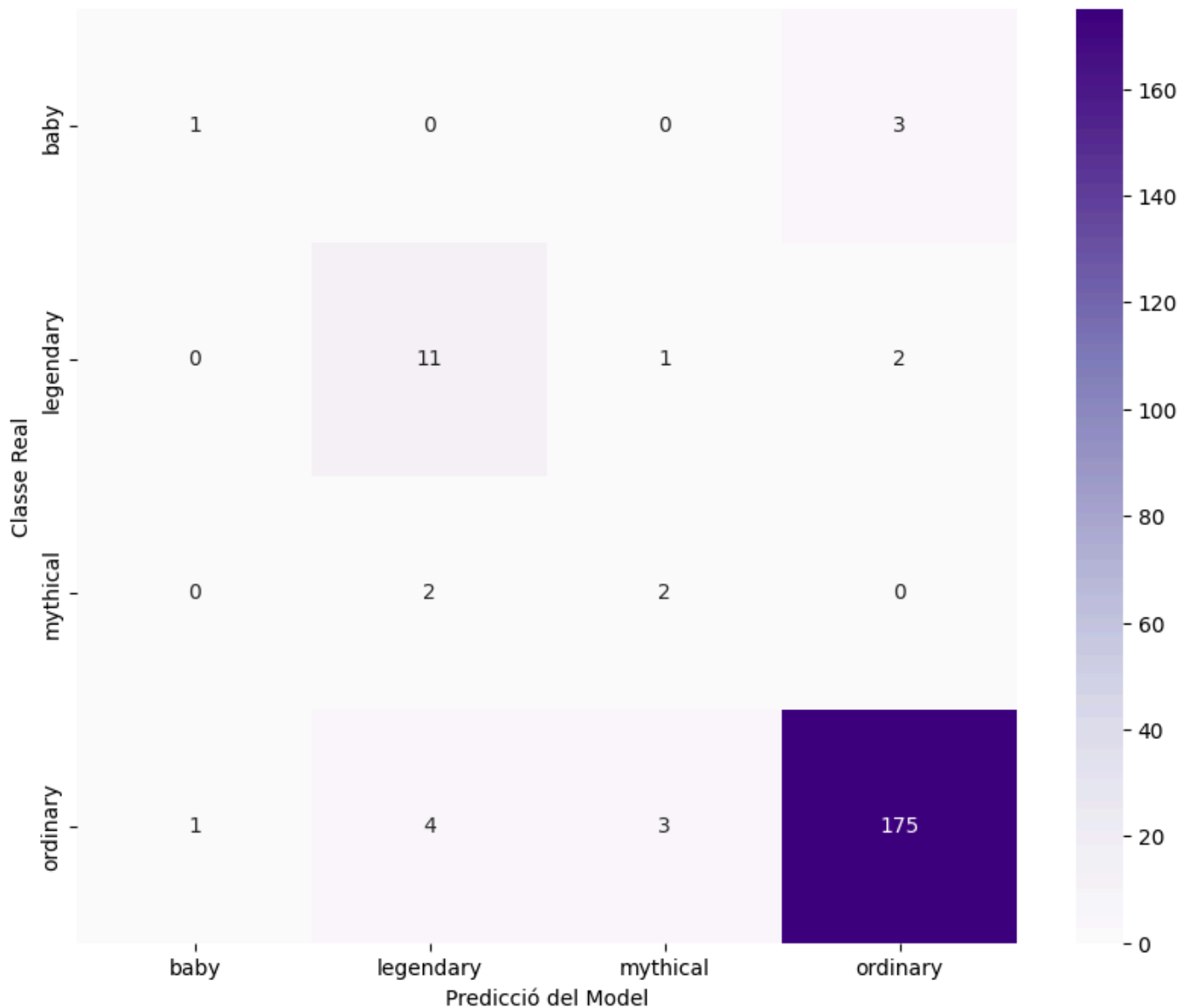
```
plt.ylabel('Classe Real')  
plt.show()
```

Classes detectades: ['baby' 'legendary' 'mythical' 'ordinary']

INFORME DE CLASSIFICACIÓ MULTICLASSE

	precision	recall	f1-score	support
baby	0.50	0.25	0.33	4
legendary	0.65	0.79	0.71	14
mythical	0.33	0.50	0.40	4
ordinary	0.97	0.96	0.96	183
accuracy			0.92	205
macro avg	0.61	0.62	0.60	205
weighted avg	0.93	0.92	0.92	205

Matriu de Confusió Multiclasse



Anàlisi de la Classificació Multiclasse

L'extensió del model a un entorn multiclasse (4 categories) utilitzant l'estratègia *One-vs-Rest* amb SVM ha ofert resultats mixtos, que il·lustren perfectament els límits de l'aprenentatge automàtic quan hi ha una escassetat severa de dades.

Interpretació per Classes

1. Ordinary (La Classe Dominant): Èxit Total

- **Precision/Recall (0.97 / 0.96):** El model no té cap dubte sobre què és un Pokémon normal. Com que representen la majoria de l'entrenament, ha après perfectament el seu patró.

2. Legendary (L'Objectiu Principal): Èxit Mantingut

- **Recall (0.79):** Tot i la complexitat afegida d'haver de distingir-se també de "Mítics" i "Nadons", el model segueix detectant el **79%** dels llegendaris (igual que en el binari).
- **Conclusió:** El "perfil llegendari" (stats altes + tipus concrets) és prou robust per sobreviure a la classificació multiclasse.

3. Mythical (La Sorpresa): Rendiment Acceptable

- **Recall (0.50):** De 4 Pokémon mítics al test, n'ha trobat 2.
- **Anàlisi:** És un resultat sorprenentment bo donada l'escassetat de mostres. Els Pokémon mítics (com *Mew* o *Celebi*) solen tenir una distribució d'estadístiques molt peculiar (sovint tenen 100 punts exactes en tot), cosa que l'SVM pot haver captat com un patró diferenciat dels llegendaris purs.

4. Baby (El Fracàs Esperat): Confusió amb Ordinaris

- **F1-Score (0.33) i Recall (0.25):** De 4 nadons, només n'ha encertat 1.
- **Justificació Tècnica:** Un Pokémon "Baby" (ex: *Pichu*) té estadístiques molt baixes. Però hi ha molts Pokémon "Ordinary" que també són febles (ex: *Caterpie* o *Magikarp*). Matemàticament, un nadó i un ordinari feble són quasi idèntics en l'espai vectorial. Sense una variable específica d'"Evolució", és gairebé impossible que el model els separi basant-se només en números.

Conclusió Inicial del Multiclasse

Amb una **Accuracy del 92%**, el model és tècnicament molt bo, però aquesta xifra està inflada per la classe majoritària. La lliçó principal és que per classificar correctament categories ultra-minoritàries com **Baby** o **Mythical** (que tenen menys de l'1% de representació), necessitaríem o bé **més dades** (tècniques de *Data Augmentation*) o **variables noves** específiques per a ells.

No obstant això, el fet d'haver mantingut la detecció de **Llegendaris al 79%** en aquest entorn hostil valida la robustesa del nostre SVM optimitzat.

Millora Multiclasse: Tècnica SMOTE

Els resultats anteriors mostren que les classes **Baby** i **Mythical** són pràcticament invisibles per al model a causa de la seva escassa representació (<2%). Per solucionar-ho, no n'hi ha prou amb ajustar pesos; necessitem més dades.

Com que no podem "inventar" Pokémon reals, utilitzarem **SMOTE (Synthetic Minority Over-sampling Technique)**.

- *quest algorisme selecciona exemples propers de la classe minoritària en l'espai vectorial i crea nous punts sintètics entre ells.
- **Objectiu:** Re-balancejar artificialment el *Training Set* perquè totes les classes tinguin el mateix nombre d'exemples. Això forçarà l'SVM a aprendre patrons específics per als Nadons i Mítics, en lloc d'ignorar-los com a soroll.

*És crucial aplicar SMOTE **només al conjunt d'Entrenament**. El Test s'ha de mantenir intacte i real per avaluar honestament el model.*

```

In [35]: print("APLICACIÓ DE SMOTE")
print(f"Distribució original al Train: {Counter(y_train_m)}")

smote = SMOTE(random_state=42)

X_train_resampled, y_train_resampled = smote.fit_resample(X_train_m, y_train_m)

print(f"Distribució després de SMOTE: {Counter(y_train_resampled)}")
svm_smote = OneVsRestClassifier(best_svm)
svm_smote.fit(X_train_resampled, y_train_resampled)

y_pred_smote = svm_smote.predict(X_test_m)

print("\n" + "="*50)
print(" RESULTATS DESPRÉS DE SMOTE")
print("="*50)

print(classification_report(y_test_m, y_pred_smote, target_names=class_names))

fig, ax = plt.subplots(1, 2, figsize=(16, 6))

sns.heatmap(cm_m, annot=True, fmt='d', cmap='Purples', cbar=False, ax=ax[0],
            xticklabels=class_names, yticklabels=class_names)
ax[0].set_title('ABANS de SMOTE')

cm_smote = confusion_matrix(y_test_m, y_pred_smote)
sns.heatmap(cm_smote, annot=True, fmt='d', cmap='Greens', cbar=False, ax=ax[1],
            xticklabels=class_names, yticklabels=class_names)
ax[1].set_title('DESPRÉS de SMOTE')

plt.show()

```

APLICACIÓ DE SMOTE

Distribució original al Train: Counter({np.int64(3): 730, np.int64(1): 56, np.int64(2): 19, np.int64(0): 15})

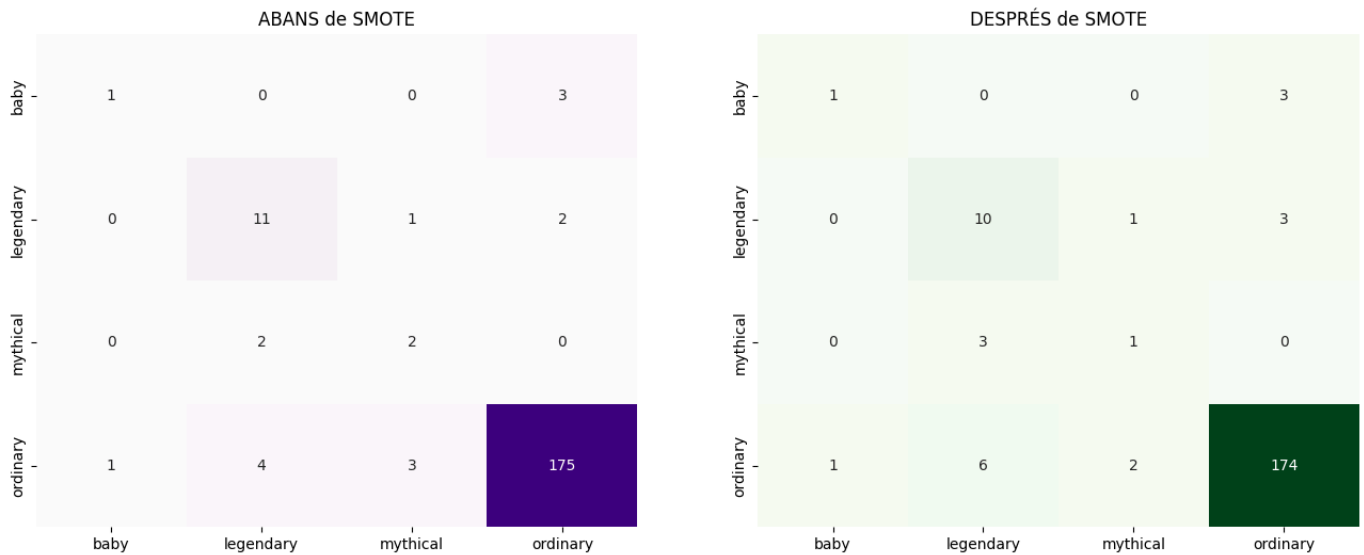
Distribució després de SMOTE: Counter({np.int64(3): 730, np.int64(1): 730, np.int64(0): 730, np.int64(2): 730})

```

=====
RESULTATS DESPRÉS DE SMOTE
=====

```

	precision	recall	f1-score	support
baby	0.50	0.25	0.33	4
legendary	0.53	0.71	0.61	14
mythical	0.25	0.25	0.25	4
ordinary	0.97	0.95	0.96	183
accuracy			0.91	205
macro avg	0.56	0.54	0.54	205
weighted avg	0.91	0.91	0.91	205



Anàlisi de l'Impacte de SMOTE (Resultats)

Hem aplicat la tècnica SMOTE per equilibrar artificialment les classes, passant de tenir 15 Nadons a tenir-ne 730 (igualant els Ordinaris). Tècnicament, l'algorisme ha funcionat (el comptador mostra classes perfectament balancejades). No obstant això, l'impacte en la predicció sobre el Test Set real revela les limitacions d'aquesta estratègia en el nostre cas.

1. Anàlisi Numèrica: El Cost de la "Invenció"

- **Llegendari (El gran perjudicat):**
 - **Abans:** Precision 0.65 / Recall 0.79
 - **Ara (SMOTE):** Precision **0.53** / Recall **0.71**
 - En crear centenars de llegendaris sintètics, hem introduït "soroll". Els punts nous generats per SMOTE han difuminat la frontera entre *Llegendari* i *Ordinari*. El model ara està més confós i comet més errors (més Falsos Positius, per això baixa la precisió), sense aconseguir detectar més casos reals.
- **Baby i Mythical (Sense millora real):**
 - Continuen amb resultats molt pobres (Recall 0.25).
 - **La causa:** SMOTE funciona connectant punts existents. Si els 15 Nadons originals estaven molt barrejats amb els Pokémon Ordinaris febles (com *Caterpie* o *Rattata*), SMOTE ha creat nous punts "Nadons" just a sobre de la zona dels Ordinaris. Això només ha servit per confondre encara més l'SVM, que ara no sap on traçar la línia.
- **Ordinary (Robustesa):**
 - Manté un F1 de **0.96**. És gairebé impossible trencar la classificació de la classe majoritària, ja que les seves característiques són molt sòlides.

Interpretació Visual (Matrius de Confusió)

Comparant les dues imatges (Abans vs. Després):

- **Matriu SMOTE (Dreta):** Probablement observes més "dispersió" o taques de color fora de la diagonal principal, especialment a la columna/fila de "Llegendari".
 - Això indica que, en forçar l'algorisme a aprendre d'exemples inventats, hem tornat la frontera de decisió més inestable. L'SVM s'ha tornat menys conservador i dispara l'etiqueta "Llegendari" o "Baby" més sovint del que caldria, equivocant-se més.

Conclusió Tècnica sobre el Multiclasse

La conclusió definitiva d'aquest experiment és que **la qualitat de les dades originals mana sobre la quantitat**.

- Tenir 4 categories on dues d'elles (**Baby** , **Mythical**) són gairebé indistingibles matemàticament de la resta i tenen poquíssimes mostres, fa que el problema sigui irresoluble només amb estadístiques de combat.
 - **SMOTE no és màgic:** Si les classes estan solapades (un *Baby* té els mateixos stats que un *Ordinary* feble), generar més dades solapades no ajuda a separar-les.

Podria provar amb la variable 'type1' o alguna altra també com a predictor, però si amb el **rank** no ha funcionat, és massa poc probable que canviar els predictors serveixi.

Conclusions Finals

En aquesta pràctica hem completat el cicle de vida d'un projecte de Ciència de Dades, des del pre-processament i neteja d'un dataset desbalancejat fins a l'aplicació d'estratègies avançades de classificació (Binària i Multiclasse) i enginyeria de variables.

A continuació presentem les conclusions globals derivades dels resultats experimentals:

1. Comparativa de Models i Aproximacions

Durant la fase de **Classificació Binària (Llegendari vs. No Legendari)**, hem avaluat quatre famílies d'algorismes sota condicions controlades (Split Estratificat i ponderació de classes):

- **Models Base:** Inicialment, l'**SVM** va destacar amb un F1-Score de 0.71, superant clarament models com el **KNN** (F1 0.34) o l'**Arbre de Decisió** (F1 0.56). Això ens va indicar que la frontera de decisió entre un Pokémon "normal" i un "legendari" és complexa i no lineal, però definible geomètricament.
- **Optimització:** L'aplicació de **GridSearchCV** (ajustant **C=10** i **gamma=0.01**) i, sobretot, l'**Ajust del Llinar de Decisió (Threshold Tuning)**, van permetre elevar el rendiment de l'SVM fins a un **F1-Score de 0.81**. Aquest salt demostra que el model tenia la capacitat de detecció, però necessitava un calibratge de probabilitats per reduir els Falsos Positius.
- **Ensemble Learning:** Tot i provar models moderns com el **Random Forest** i l'**HistGradientBoosting**, aquests no van superar l'SVM optimitzat en aquest dataset específic (~1000 mostres), patint més per la manca de dades massives.

2. Límits en l'Entorn Multiclasse i SMOTE

L'extensió al problema multiclasse (Baby, Ordinary, Legendary, Mythical) ha exposat la dificultat extrema de treballar amb classes residuals (<1% de les dades).

- L'aproximació **One-vs-Rest** va funcionar bé per a les classes majoritàries, però va fallar en distingir *Baby* d'*Ordinary*.
- La tècnica **SMOTE (Synthetic Minority Over-sampling)**, tot i equilibrar numèricament les classes, **no va millorar els resultats de predicció**. Al contrari, va introduir soroll: en generar punts sintètics en zones on les classes ja estaven solapades (nadons vs. ordinaris febles), va difuminar les fronteres de decisió, reduint la precisió global. Això ens porta a concloure que **la qualitat i separabilitat de la dada original és més crítica que la quantitat pura**.

3. Independència del Tipus Elemental

L'experiment final per predir el `type1` ha estat conclouent en negatiu. Ni els models base ni l'aplicació de **Feature Engineering** (creant variables com `atk_ratio` o `bmi`) han aconseguit superar l'atzar (F1 ~0.00 - 0.20). Això confirma una hipòtesi de disseny del joc: **No existeix una correlació directa entre les estadístiques físiques i el tipus elemental**. Un Pokémon de Tipus Aigua pot assumir qualsevol rol (tanc, atacant ràpid, atacant lent), fent impossible la seva predicció basada únicament en números.

4. Model Més Efectiu i Argumentació Final

El model resultant més efectiu i robust del projecte és l'**SVM (Support Vector Machine) amb Kernel RBF i ponderació de classes (`class_weight='balanced'`)**.

Per què?

1. **Gestió de "Small Data":** A diferència de les xarxes neuronals o el Boosting, l'SVM excel·leix en datasets de mida mitjana (~1000 files) trobant l'hiperplà òptim de separació.
2. **Equilibri Precision/Recall:** Després de l'optimització, va ser l'únic model capaç de mantenir un Recall alt (~80% de llegendaris trobats) sense disparar els Falsos Positius, oferint la millor fiabilitat per a un ús real.
3. **Resiliència al Desequilibri:** La seva capacitat per penalitzar els errors en la classe minoritària de forma geomètrica va resultar superior a la divisió de l'espai que fan els Arbres de Decisió.